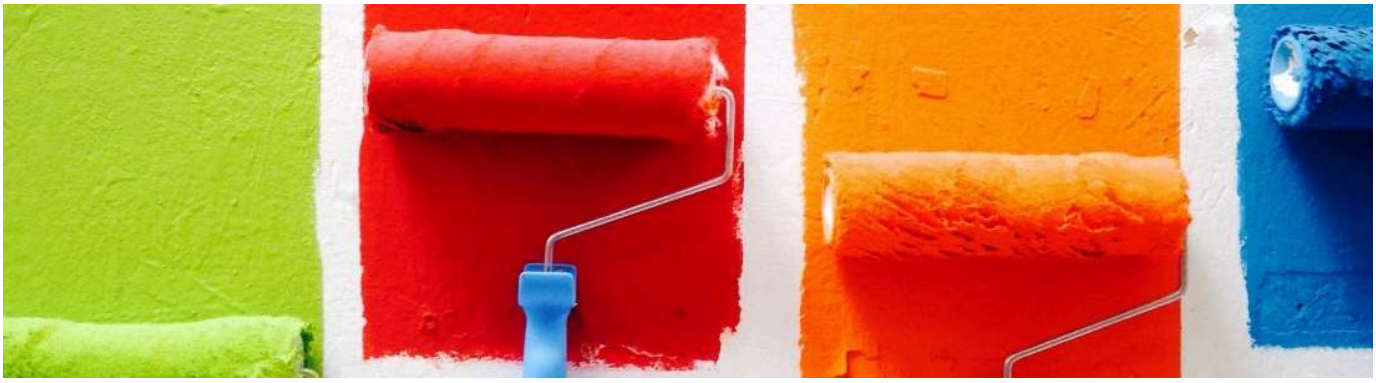
6. Overview and concepts

Next



K-Means clustering

K-means clustering is a data mining method aiming to uncover hidden, natural groupings (i.e. clusters) in data. Data points belonging to the same cluster tend to share similar characteristics. They may correspond to such instances as customers with similar purchasing behaviour, plants belonging to the same species, or a subpopulation with a similar sociodemographic profile. Further, clusters discovered by k-means may also be used for prediction or classification.

K-means is well suited for exploratory data analysis and can be applied to various data types, including images and texts. This week, you will learn how to implement this data mining model in Python. You will also learn some standard techniques for optimising its clustering performance, in addition to visualising the discovered clusters.

Algorithm overview

The k-means clustering method iteratively finds k number of data points that serve as the **means** summary (hence the name "k-means") of all data points that belong to the same clusters. These data points are called **centroids**, and the value k must be chosen by the data analyst before implementing the model.

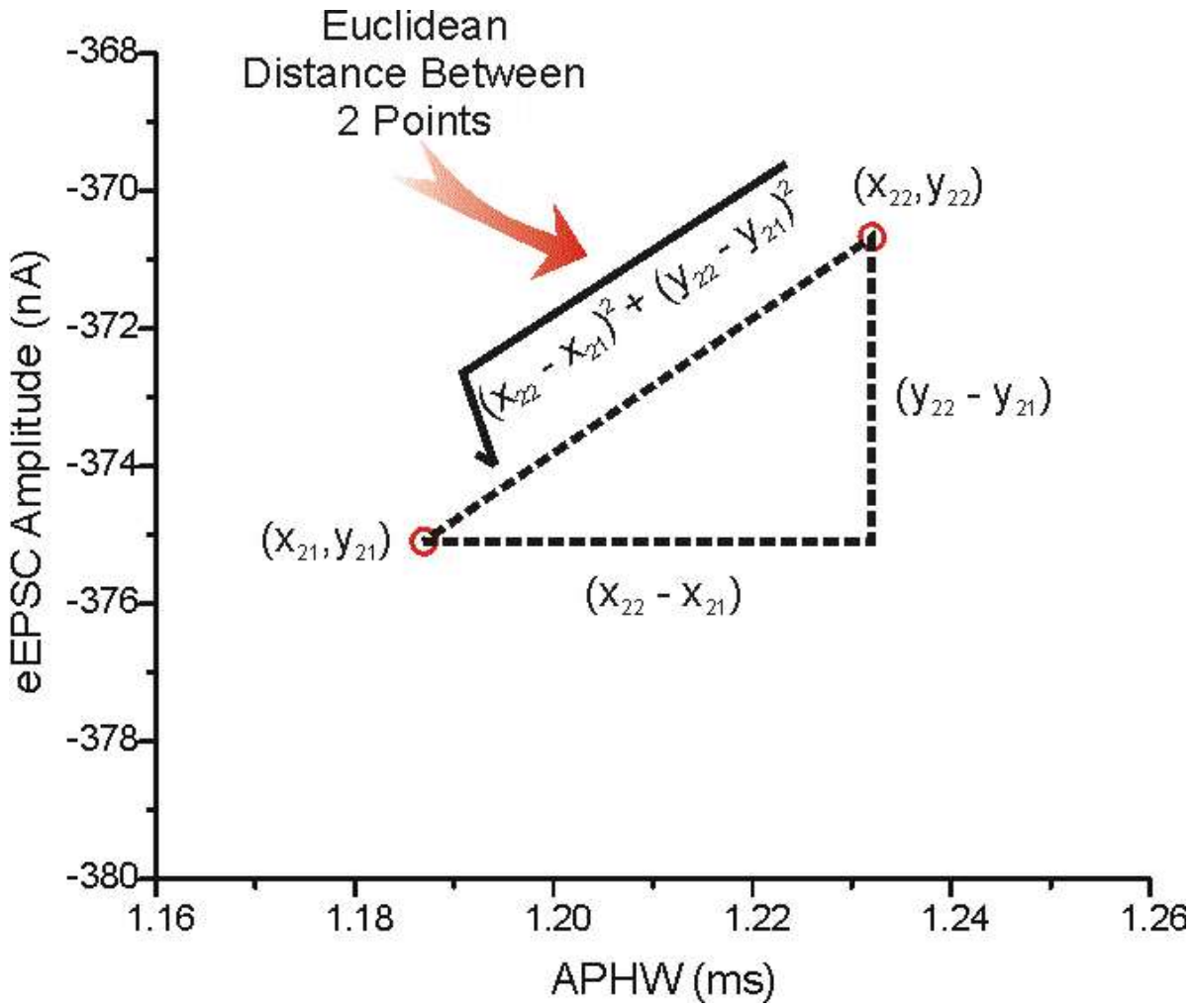
More specifically, k-means works according to the following steps:

1. `k` number of centroids are created randomly within the Euclidean space of a given dataset. There are several approaches for initially choosing the centroids. The k-means implementation in `scikit-learn` uses the **k-means++** initialisation method to achieve fast clustering.
2. Each data point in a dataset is then assigned to the nearest `k` centroids. The distance between a data point and a centroid is calculated as the **Euclidean distance** between them. In effect, this step minimises the total distance between all data points to their respective centroids.
3. k-means then recalculates the position of all `k` centroids by taking the means of attribute values of all data points belonging to each centroid.
4. Repeat Steps 2 and 3 until some criteria are met. These criteria include: the maximum number of iterations allowed, the minimum sum of distances between the data points and their corresponding centroid, or no significant changes to the positions of the centroids between two consecutive iterations.

As mentioned in Step 2 above, an essential equation in the k-means algorithm is the Euclidean distance *d*:

$$d(p,q) = \sqrt{\sum_{i=1}^{n} (p_i - q_i)^2}$$

, where *p* and *q* are two points in Euclidean *n*-dimensional space, and p_i and q_i are Euclidean vectors. The following illustration shows a 2-dimensional Euclidean space (n=2). Let point *p* be denoted by 2-dimensional coordinate (x_{21}, y_{21}) and point *q* be denoted by coordinate (x_{22}, y_{22}). The Euclidean distance between the points is therefore the length of a simple straight line connecting the two points.



Source: hlab.stanford.edu (https://hlab.stanford.edu/brian/making_measurements.html)

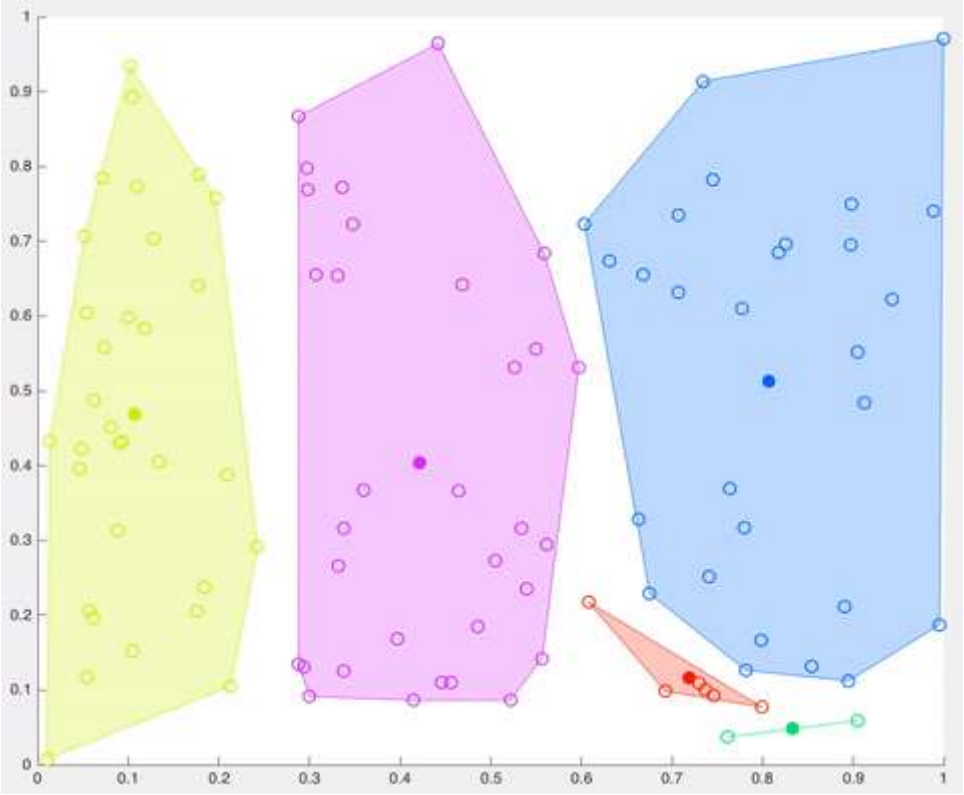
Here again, we encounter linear algebra. Note that the straight line between the two points above is a vector. The Euclidean distance is a mere adaptation of the L2 norm (https://machinelearningmastery.com/vector-norms-machine-learning/#:~:text=The%20L2%20norm%20calculates%20the,is%20a%20positive%20distance%20value.) of a vector in linear algebra.

The original L2 norm equation calculates the distance of a vector, let's say $x = (x_1, x_2, x_3)$, from the *origin* of the vector space (i.e. a vector of zero values):

$$
\begin{aligned}
|x| &= \sqrt{x_1^2 + x_2^2 + x_3^2} \\
&= \sqrt{(x_1 - 0)^2 + (x_2 - 0)^2 + (x_3 - 0)^2} \\
&= \sqrt{\sum_{i=1}^{n} (x_i - 0)^2} \\
&= d(x, 0)
\end{aligned}
$$

Euclidean distance is a simple extension of the L2 norm, which replaces the *origin* with the coordinate of any given initial point.

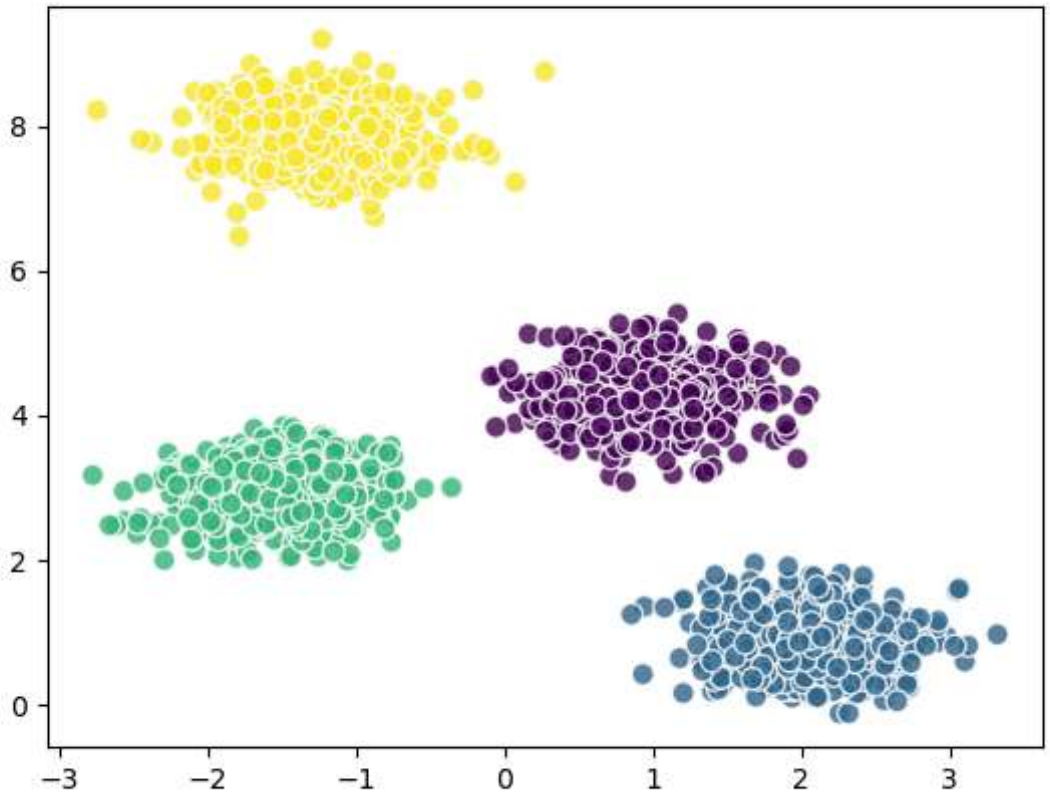
Below is a simulation of the effect after each k-means iteration. A cluster is represented by a colour-shaded convex hull. There are five clusters in this visualisation, each represented by a distinct colour. The centroid is the non-hollow data point at the centre of each cluster. After 11 iterations, we can observe that the position of each centroid no longer changes, suggesting an optimal clustering configuration has been reached and that the clustering process should terminate.



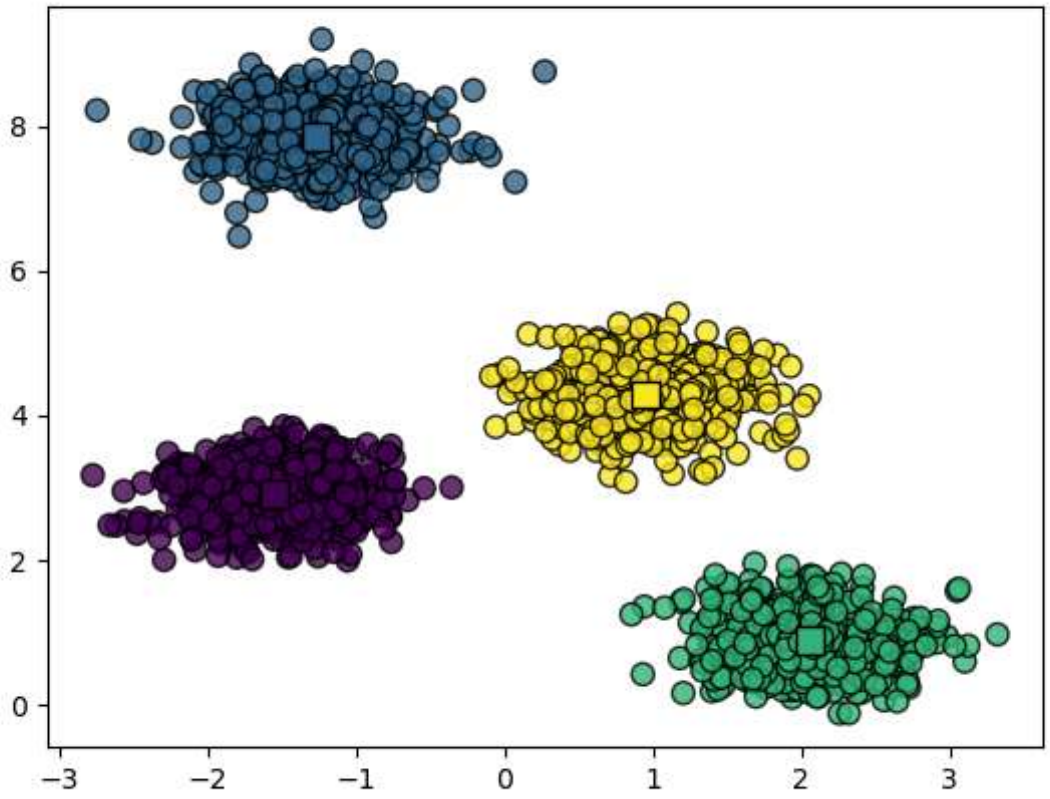
Source: KDNuggets (https://www.kdnuggets.com/2019/05/guide-k-means-clustering-algorithm.html#:~:text=K%2Dmeans%20allocates%20every%20data,centroid%20than%20any%20other%20centroid)

Example 1

The first example below visualises a synthetic dataset with 4 naturally-occurring clusters of roughly spherical shapes.



Running k-means clustering (k=4) on the dataset shows four cluster centroids that correspond very well to the natural groupings of the data (in the image below, the centroids are the squares surrounded by other data points). This suggests that k-means produce good clusters.

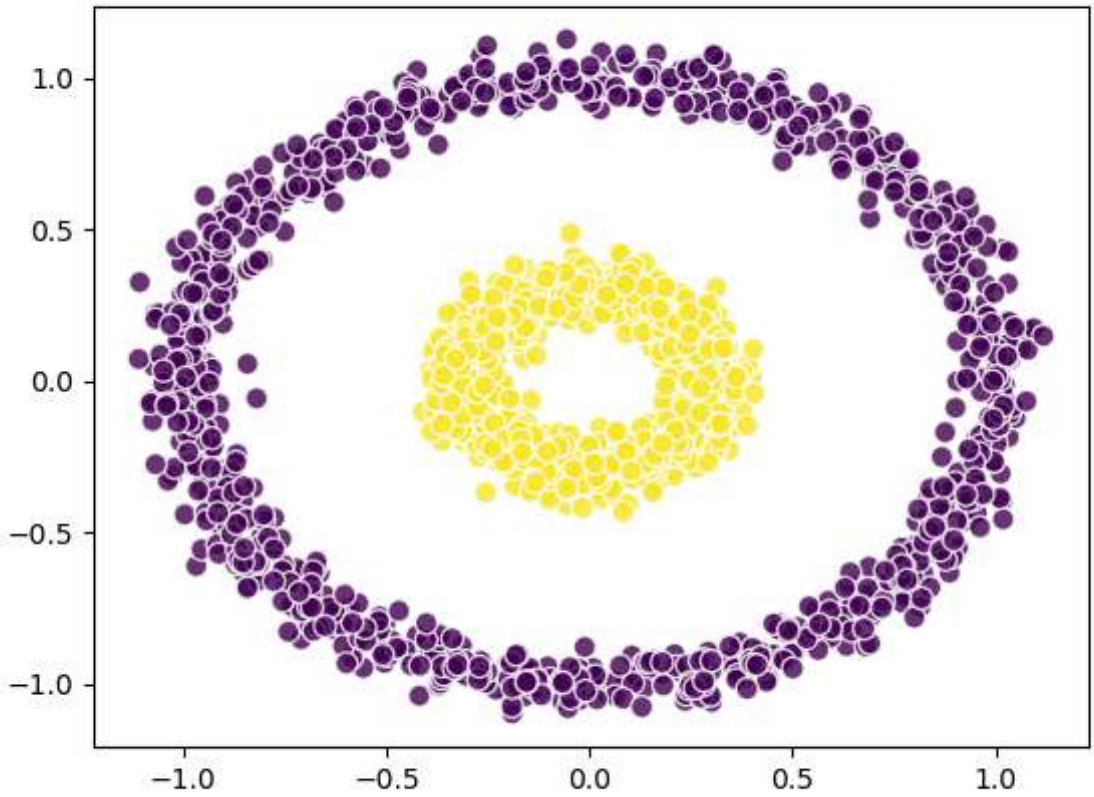


Python code:

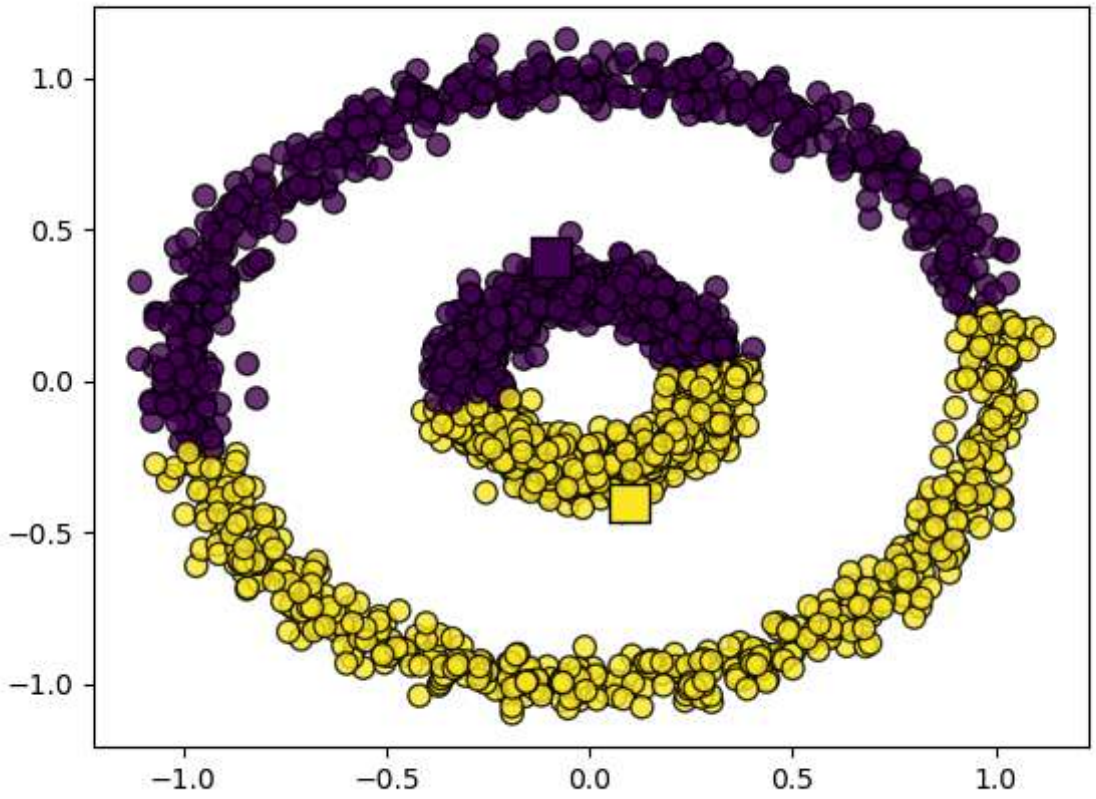
kmeans_blobs.py

Example 2

The second example illustrates a scenario where k-means is limited in its ability to detect good clusters. Below is another synthetic dataset generated to resemble two circles; one encircling the other.



People could easily perceive the two naturally occurring circular-shaped clusters in the dataset: (1) a set of purple data points that form the larger circle, (2) another set of yellow data points that make up the smaller circle. Detecting the same clusters, unfortunately, proves to be quite challenging to k-means, as illustrated below.



Python code:

kmeans_circles.py

Limitations

The previous example underlines some limitations of the k-means clustering algorithm:

1. First, the model strives to produce clusters with **uniform sizes**, even when the actual clusters do not necessarily have similar sizes.
2. K-means also assumes that data points surrounding a centroid are constrained within a **spherical area**. Consequently, when the shape of the true clusters does not conform to such a spherical assumption, the model is likely to perform poorly, as perfectly demonstrated in dealing with the circular-shaped clusters.
3. Finally, the algorithm is also sensitive to the presence of **outliers** and noises in data. Consequently, applying outlier detection and removal before running k-means is sometimes recommended.

Further readings

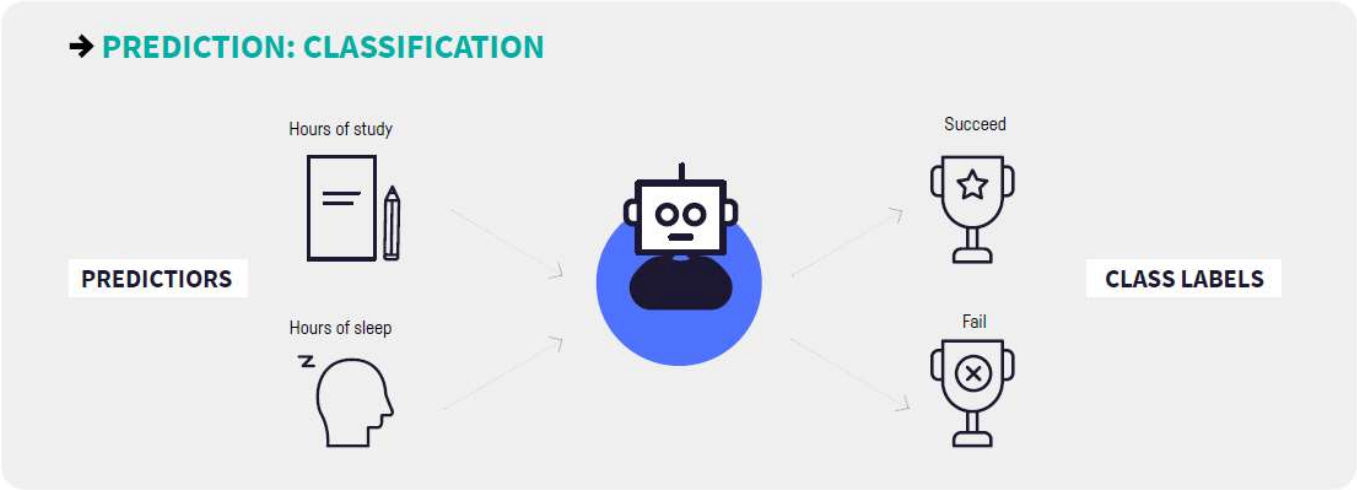
Yse, D.L. 2019. A complete guide to K-Means clustering algorithm. KDNuggets. (https://www.kdnuggets.com/2019/05/guide-k-means-clustering-algorithm.html#:~:text=K%2Dmeans%20allocates%20every%20data,centroid%20than%20any%20other%20centroid)

7. Overview and concepts

Next

Classification

Unlike regression, classification refers to a prediction task where the value to be predicted is nominal or categorical. For instance, we may want to predict whether students "succeed" or "fail" (categorical values) based on other data that we know (e.g. hours of study, hours of sleep), illustrated below:



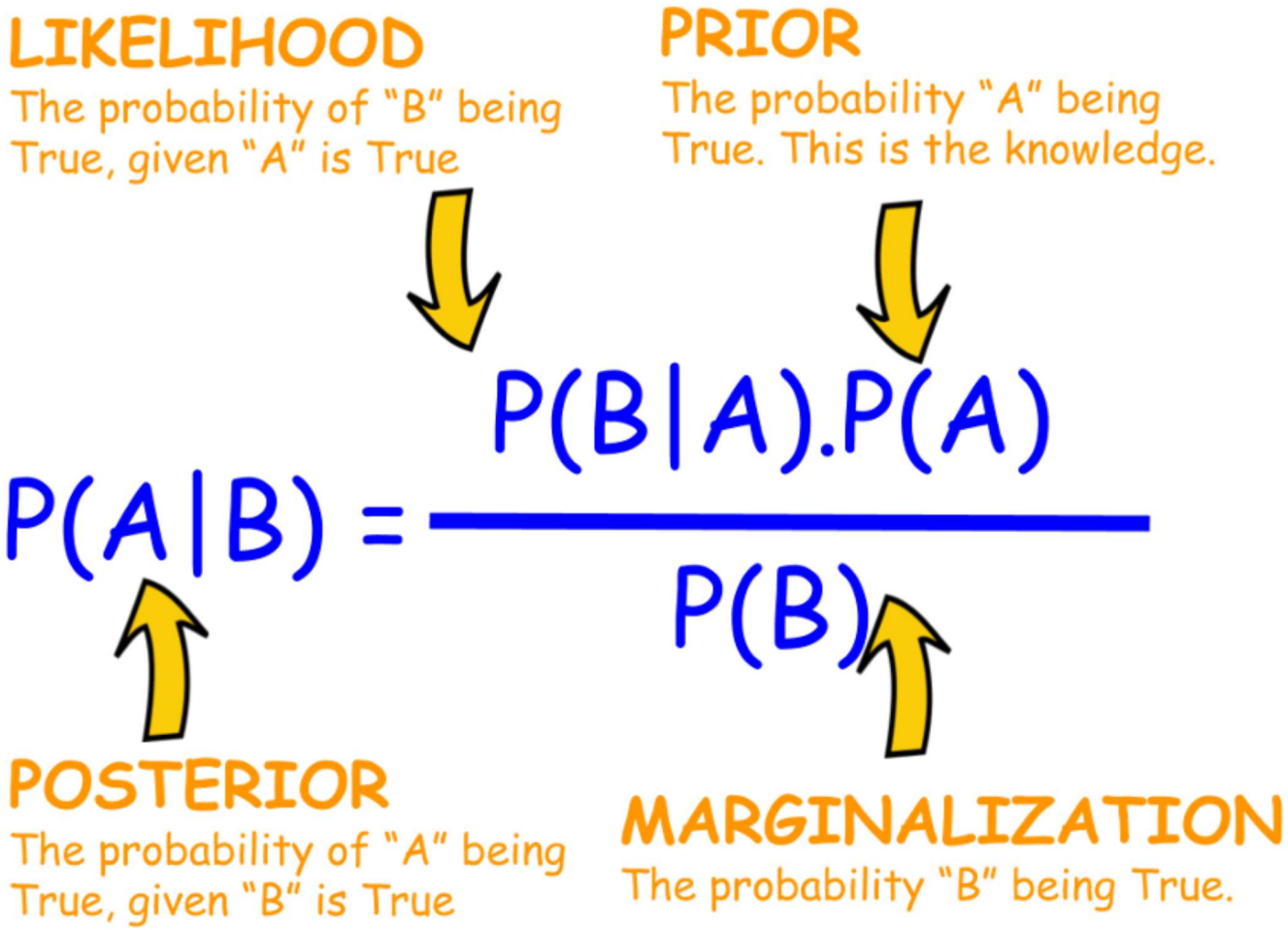
Source: dataiku.com (https://pages.dataiku.com/machine-learning-basics-illustrated-guidebook)

Naive Bayes algorithm

The Naive Bayes classifier is one of the simplest but potentially powerful classifiers. It is based on the Bayes Rule.

The Bayes Rule

The Bayes Rule (the Bayes theorem or the Bayes formula) is one of the key principles in the theory of probability. It is particularly useful for calculating the probability of events marked by many uncertainties. The rule is a derivation of the conditional probability formula and has the following anatomy:



Source: towardsdatascience.com (https://towardsdatascience.com/bayes-rule-with-a-simple-and-practical-example-2bce3d0f4ad0)

Refer to the section **Bayes Rule derivation** of the learning materials this week if you are interested to know how we derive the Bayes Rule from the Conditional Probability equation.

Naive Bayes classifier

Unlike the Linear Regression algorithm which predicts numerical output values, Naive Bayes is typically used for predicting nominal/categorical output values. These nominal or categorical values are often called '*class*', such that the process of predicting them is termed **classification**.

Assuming that a dataset is in a tabular format with rows and columns, the Naive Bayes algorithm essentially works in two steps, as follows:

Step 1: calculate the probability of a class $\boldsymbol{c_j}$ given the input values $\boldsymbol{X}$ from a row (posterior probability). This is the *probability estimation* step.

This step involves a generalisation of the Bayes Rule above. Let c_j represent a specific class (a categorical value of the response variable) and $\boldsymbol{X}$ represent a set of explanatory variables' values, we replace A and B in the previous Bayes Rule with c_j and X, respectively:

$$P\left(c_j|X\right) = \frac{P\left(X|c_j\right) \cdot P\left(c_j\right)}{P(X)}$$

Next, assume that the response variable has at least two classes (because it is not meaningful to perform classification on a response variable with only one categorical value), the Naive Bayes classifier must calculate the posterior probability of the first class c_1 given X, $P(c_1|X)$, and the probability of the second class c_2 given X, $P(c_2|X)$. In this calculation, X are the same *regardless of the class.*

Since we are only interested to know how the classes' posterior probabilities compare relative to each other (i.e. which has the higher or lower probability score), it is safe to remove the common denominator *P(X)* from the Bayesian equation to simplify and speed up its calculation:

$$P\left(c_j|X\right) = P\left(X|c_j\right) \cdot P\left(c_j\right)$$

Also, because X is made up of a set of explanatory variables' values that occur together (i.e. joint probability), $X=(x_1, x_2, x_3, ..., x_i)$, we can further expand the formula:

$$\begin{aligned} P\left(c_j|X\right) &= P\left(x_1, x_2, x_3, \ldots, x_i|c_j\right) \cdot P\left(c_j\right) \\ &= P\left(x_1|c_j\right) \cdot P\left(x_2|c_j\right) \cdot P\left(x_3|c_j\right) \cdots P\left(x_i|c_j\right) \cdot P\left(c_j\right) \end{aligned}$$

The expansion above is possible given the naive assumption that the variables in X are independent of each other (see probability of independent events (https://brilliant.org/wiki/probability-independent-events/)).

Finding the posterior probability of a class is equivalent to asking the question "*What is the probability that a particular object belongs to a class given its observed feature values*?". For example, we may ask "What is the probability that a person has diabetes (c_1) or no diabetes (c_2) given a certain value for a pre-breakfast blood glucose measurement (x_1) and a certain value for a post-breakfast blood glucose measurement (x_2)?"

Step 2: once all posterior probabilities are calculated, the last step is to select a class with the highest probability score as the true class for each row in the data. This is the *classification* step.

Here, we show how to implement a Naive Bayes classifier above manually by constructing conditional probability tables in Microsoft Excel. These tables embody the equation above. This activity should give a better understanding of the workings of this classifier.

Example
The golf weather dataset consists of weather-related categorical explanatory variables for predicting whether to **play** or **not to play** golf. Construct the probability tables based on the following training set:

weather_train.xlsx

Using the conditional probability tables, perform classification on this test set:

weather_test.xlsx

Solution

weather_NB_solution.xlsx

Laplace Smoothing

The above solution also illustrates a problem: if any test case incorporates a zero probability into the Naive Bayes equation, it cancels the other non-zero probabilities and nullifies the other relevant input values that should have been counted in the classification.

To solve this "zero probability problem", in the solution, we added 1 to every frequency count in the conditional probability tables.

This clever trick is called the **Laplace Smoothing** (or Additive Smoothing), invented by the French scholar Pierre-Simon Laplace. Notice how the classification accuracy improves following the Laplace Smoothing.

Naive Bayes variants

The Naive Bayes algorithm is a *generative* model that performs classification by assuming that the explanatory variables' values are generated from a certain type of probability distribution. There are a few variants of the algorithm depending on the probability distribution it assumes. Knowing them is important because different variants are suited for different types of explanatory variables we deal with.

Categorical Naive Bayes

This is the same variant as the example above. It is suitable for scenarios where all explanatory variables are non-binary categorical (discrete) variables. It assumes that data are generated from the categorical distribution (https://www.statology.org/categorical-distribution/).

Bernoulli Naive Bayes

This variant assumes that data are drawn from the Bernoulli distribution (https://brilliant.org/wiki/bernoulli-distribution/) and that each explanatory variable has only two possible outcomes or categorical values (binary). Bernoulli distribution models the following situations:

- A newborn child is either male or female. (Here the probability of a child being a male is roughly 0.5.)
- A student either passes or fails an exam.
- A tennis player either wins or loses a match.
- A dart is thrown at a circular dartboard and land randomly over its area. The dart will either land closer to the centre than to the edge or not (in the second case it is either closer to the edge or equally distant from the centre and the edge).

Multinomial Naive Bayes

The multinomial Naive Bayes applies when it is believed that the explanatory variables come from the multinomial distribution (https://stattrek.com/probability-distributions/multinomial.aspx). This variant is suitable when each explanatory variable comprises more than two categorical values. Example: data from rolling a die 10 times to see what numbers we obtain.

The categorical distribution is a *special case* of the multinomial distribution with only a single trial. In some fields such as natural language processing, categorical and multinomial distributions are synonymous and it is common to speak of a multinomial distribution when referring to a categorical distribution.

Gaussian Naive Bayes

The Gaussian Naive Bayes should be used when all explanatory variables are made up of continuous values. It assumes these values arise from the Gaussian distribution (https://www.mathsisfun.com/data/standard-normal-distribution.html) (i.e. Normal distribution). The implied limitation of the model is that it may not work well if the values follow a non-Gaussian distribution, are skewed, or contain significant outliers.

Applications

The Naive Bayes algorithm is extremely fast and scalable against large datasets. It is well known to work well in such applications as email spam detection and filtering, natural language processing, and recommendation systems.

Assumptions and limitations

There are, however, limitations to the algorithm. It makes a very strong assumption about the *conditional independence* among the input variables X. It believes that the values of one input variable do not affect the values of all other input variables in a dataset (hence "Naive" in that sense), represented by this component of the equation:

$$P\left(x_1 | c_j\right) \cdot P\left(x_2 | c_j\right) \cdot P\left(x_3 | c_j\right) \cdots P\left(x_i | c_j\right)$$

Unfortunately, much of our reality does not conform with this assumption such that there are chances that the Naive Bayes algorithm will not produce accurate predictions. Also, owing to this naive assumption, it is common in practice to use the Naive Bayes algorithm only as a benchmark baseline to compare against other better classifiers.

Ironically, the same limitation above gives the algorithm its phenomenal speed. Also, even though its conditional independence assumption is often not realistic and often leads to poor probability estimation (see **Step 1** of the Naive Bayes algorithm explained above), the difference between the probability score of the true class versus that of the false class is often large enough to ensure the algorithm is capable of picking the right class (which means **Step 2** still works well). This explains why the algorithm still performs well, particularly for natural language processing tasks (such as text sentiment classification). However, the probability of certain words appearing in natural language texts is usually not conditionally independent from the presence of other words.

8. Overview and concepts

Previous Next

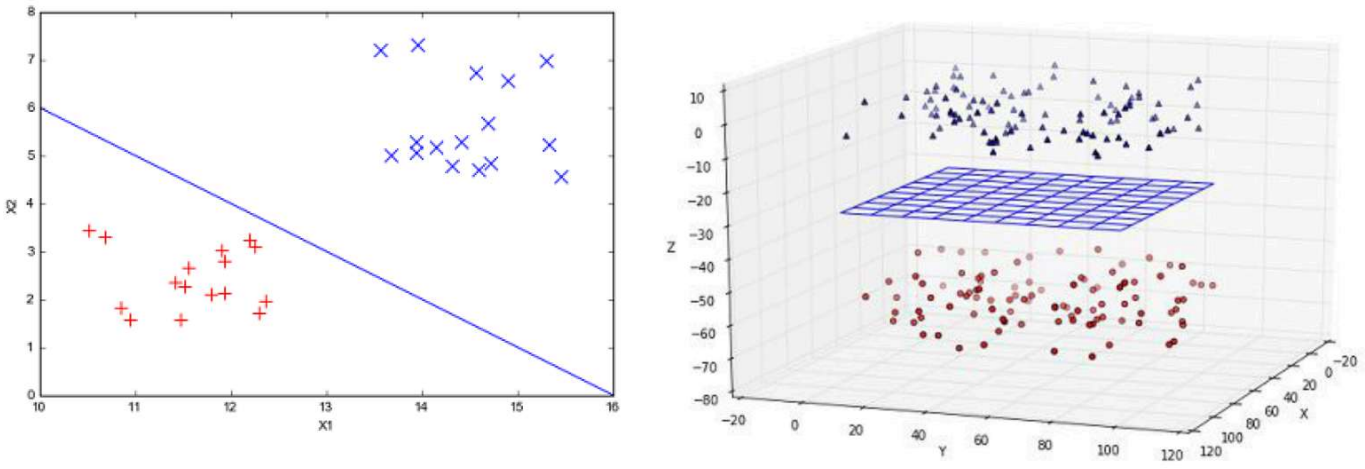
Video: The Concept of SVM

Watch this video to grasp the intuitions behind the Support Vector Machines classifier. Our workshop session will only cover practical activities.

Support Vector Machines (SVMs)

Support Vector Machines is a suite of related classification algorithms that aim to classify data using a *hyperplane* (you will also find out later that a variant of SVMs called SVR is suited for regression).

On the left plot below, the hyperplane is a straight line that separates data points belonging to two different classes, such that any point that is on one side of the hyperplane is classified as belonging to a blue class and anything on the other side as a red class.



Source: Kowalczyk, A. Support Vector Machines Succinctly.

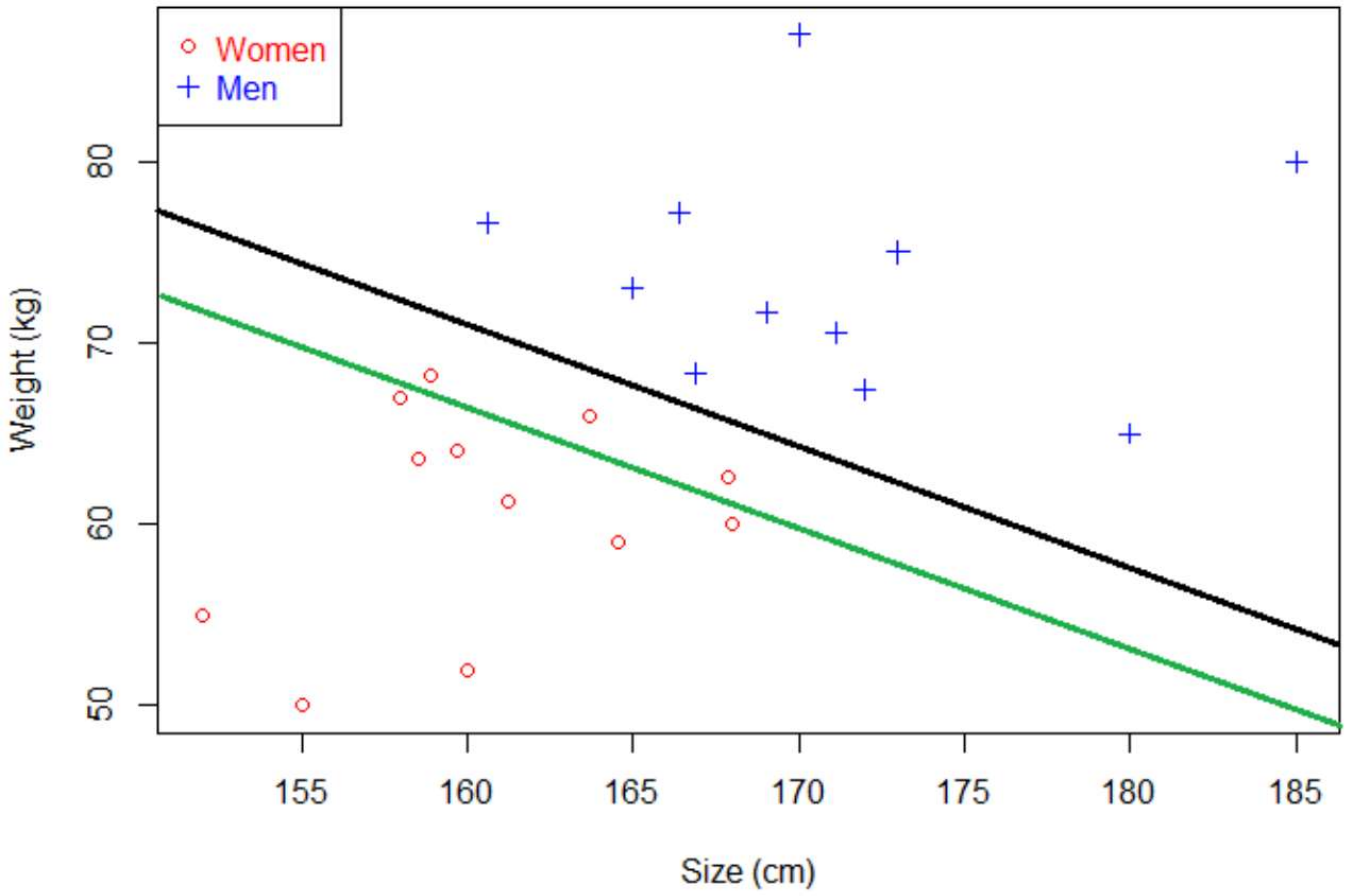
Do not be deterred by the word "hyperplane". A **hyperplane** is just a set of points whose geometric shapes are dictated by the *n*-dimensional space where it resides. In a 1-dimensional space (i.e. dataset with 1 explanatory variable), a hyperplane is a **single point**. In a 2-dimensional space (2 explanatory variables), it forms a **line**. In a 3-dimensional space, it takes the form of a geometric plane (see the right plot above). For any higher-dimensional space, it becomes difficult to visualise the corresponding hyperplanes. So, we just call them hyperplanes and work with them mathematically. From this point onwards, we explain the SVM model as a line-shaped hyperplane within a 2-dimensional space. Those explanations directly generalise to other higher-dimensional spaces.

Please note that knowing the mathematical details of the algorithm is not essential for using SVM in practice nor is it required by this unit. Before going further, I'd like to acknowledge author Alexandre Kowalczyk and his excellent free e-book Support Vector Machines Succinctly (https://www.syncfusion.com/succinctly-free-ebooks/support-vector-machines-succinctly) (highly recommended) from which much of the content here has been adapted. To those interested to know more about the maths behind SVM, please read the book and also visit the blog www.svm-tutorial.com (https://www.svm-tutorial.com/) for a quick, nice intro to SVM. In reading those, you need to know basic linear algebra, which is why we covered linear algebra earlier this semester!

Finding the optimal hyperplane

Building an SVM model is about finding the best line (i.e. hyperplane) that separates data points from different classes. It is not difficult to imagine by looking at the previous 2D plot that there are many possibilities for drawing a line that separates the blue and red classes.

For another illustration, observe the following diagram. Both the **black** and **green** lines are hyperplanes, although we can tell that the black one separates the women and men classes more clearly than the green one, according to the weight and size variables.



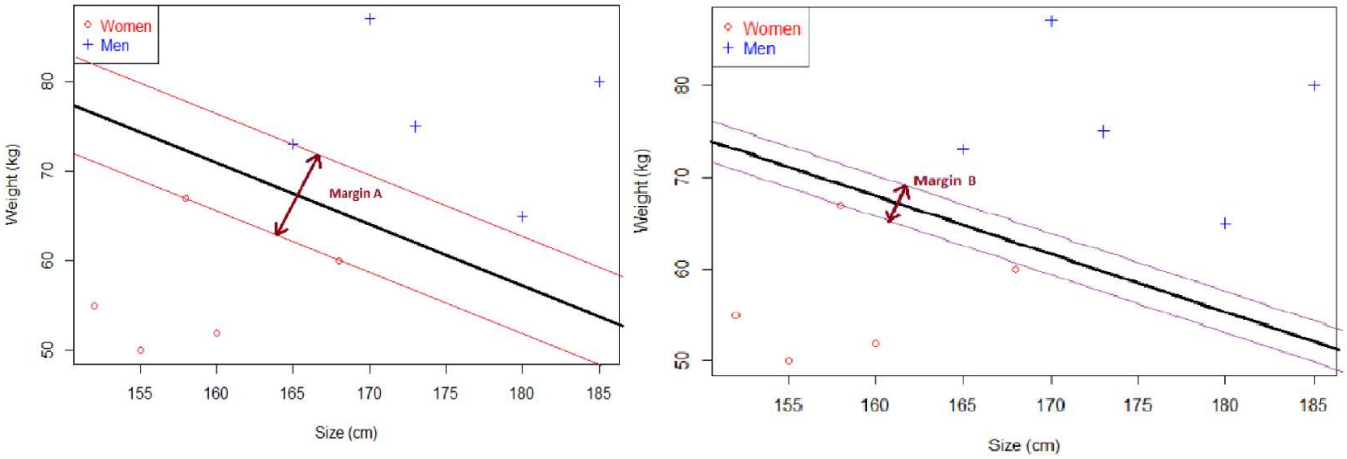
Source: svm-tutorial.com (https://www.svm-tutorial.com/2014/11/svm-understanding-math-part-1/)

SVM's way of finding the optimal or the right hyperplane hinges on an important concept called **margin**. Every hyperplane has a *margin*, the distance from the hyperplane to its closest data point (regardless of the class). For instance, in the diagram below, two hyperplanes are plotted against the same set of data points. The first hyperplane on the left plot separates the blue and red classes pretty well with a wide margin (*margin A*). In contrast, the other hyperplane in the next plot seems to fall a little too close to the red data points that its margin (*margin B*) is smaller (the margin is a distance from this hyperplane to its closest data point which happens to be a red one).

In other words:

- if a hyperplane is very close to a data point, its margin will be small.
- the further a hyperplane is from a data point, the larger its margin.

- a margin is determined by the positions of data point(s) closest to a hyperplane. These data points are called the **support vectors** (hence the name SVM).



Source: ~~svm-tutorial.com~~ (https://www.svm-tutorial.com/2014/11/svm-understanding-math-part-1/)

Importantly, it has been proven mathematically that **finding the optimal hyperplane is equivalent to finding the one that has the largest margin**.

One may further observe in the plots above that a margin is indeed the distance between a pair of hyperplanes that enclose the primary hyperplane. These are the two red lines that enclose the black one. Consequently, maximising a hyperplane margin is equivalent to maximising the distance between these two supporting red hyperplanes.

In summary, to find the largest margin:

1. We have a dataset.
2. Select two hyperplanes that separate data points with no point between them.
3. Maximize their distance (the margin)

The above summarises the basic intuition behind SVM. Finding the hyperplane with the largest margin is achieved by employing a rather sophisticated optimisation technique that satisfies some nifty mathematical constraints. As mentioned earlier, knowing those maths is not essential to most SVM users and, therefore, is not covered in this unit.

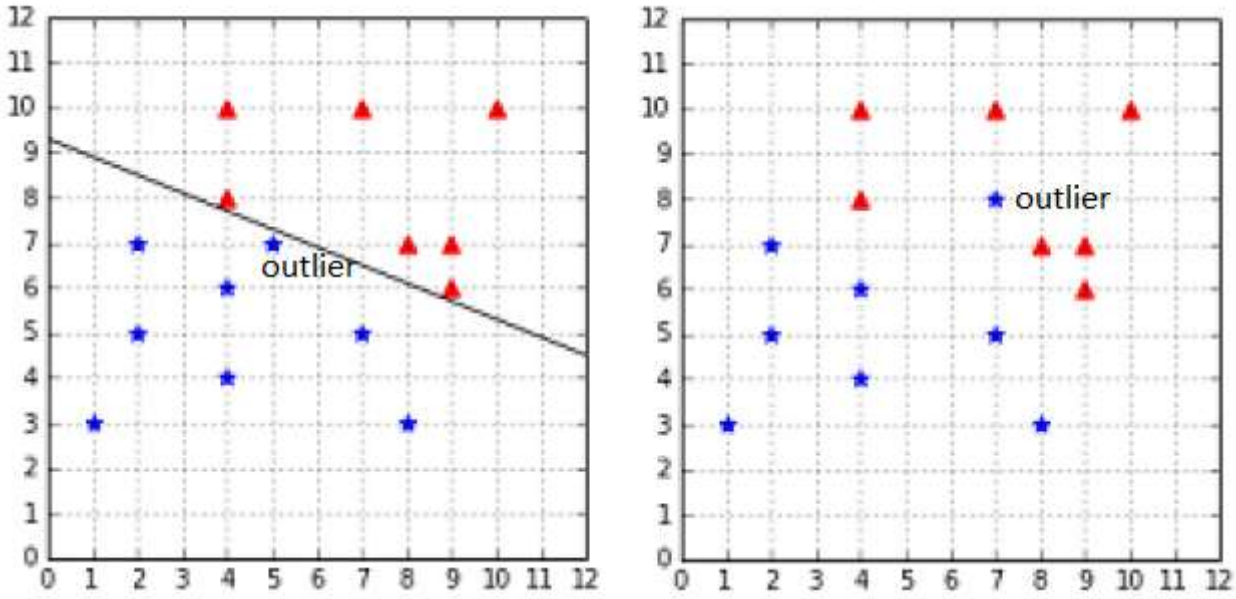
Dealing with noisy data using soft margin SVM

As indicated earlier, finding the right hyperplane for SVM is done by optimising certain mathematical equations while satisfying a set of constraints. At first, these constraints are pretty inflexible, producing the so-called "hard margin" for a hyperplane. This margin works fine if the data points used for training

an SVM classifier are linearly separable and free from unexpected noises. The problem is that many real-world datasets are not always linearly separable and may contain some outliers. In those cases, the hard margin approach may not be as effective.

It is important to understand that, historically, the original SVM optimisation approach was aimed at the total elimination of any classification errors during the training phase. This is what gives it the hard margin. But this assumption proved to be untenable for noisy data. The following plots illustrate the problem. In the left plot, there is an outlier (blue star) near SVM's decision boundary. Although it does not cause the training data to lose its linear separability (i.e. there is still a hyperplane to be found), it causes a hard margin SVM to end up with a hyperplane with a very narrow margin, owing to its effort in eliminating classification errors during training.

Granted that there is no misclassification error during the training phase as shown in the plot. However, this type of hyperplane is prone to producing many misclassification errors on the unseen test set. Slight variations in the values of explanatory variables between the training and the test set could easily confuse the class membership assignment.



Source: Kowalczyk, A. Support Vector Machines Succinctly.

The plot on the right illustrates a different problem. Here, the mere presence of the outlier (a blue star among the red triangles) causes the training data to lose its linear separability. No hyperplane can be found by a hard margin SVM for this case, rendering classification impossible.

In response to these problems, subsequent versions of SVMs were designed not to eliminate errors during the training phase but to *minimise* them. This design principle led to the formulation of "soft margin" SVMs. These are the common versions of SVMs we use nowadays. Having soft margins means SVM incorporates a specific error term into its optimisation's constraint formula when searching for the best hyperplane. In other words, when finding the right hyperplane during the training phase, a certain amount of classification errors is admissible.

Just how much error is admissible?

There is no single answer to that. Instead, researchers introduced an SVM setting or hyperparameter, called *hyperparameter C*, to give SVM users the control over how much classification error is admissible when training the model and how much penalty is applied for each error. This leads us to the concept

called *regularisation*, explained below.

Regularisation and hyperparameter C

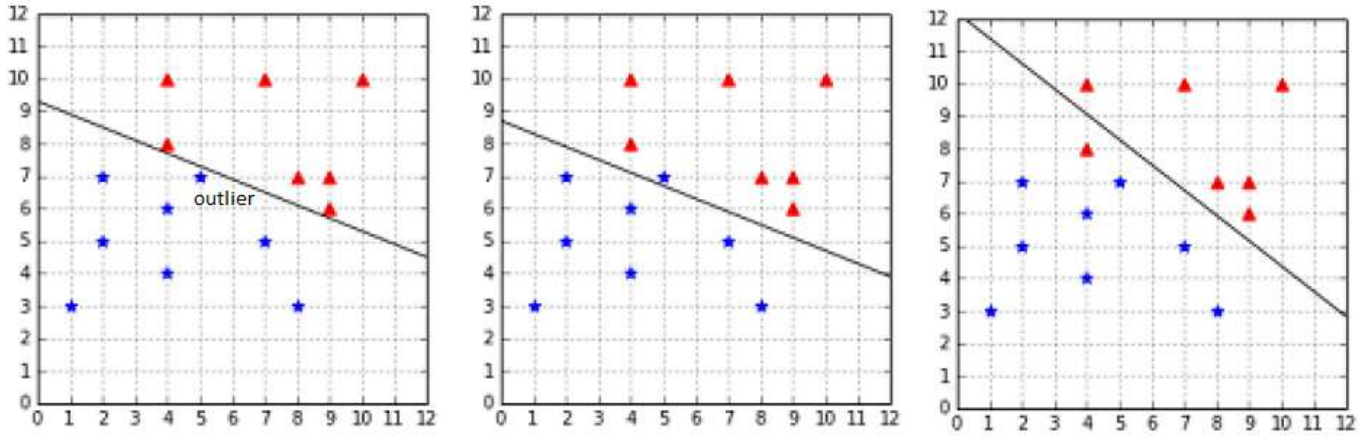
Introducing errors into soft margin SVMs is tied to the idea called **regularisation**. Regularisation provides SVM with the flexibility to relax its hard margin requirement so that it can better handle noisy or non-linearly separable data.

In machine learning, regularisation is a technique that modifies a classifier's function by introducing an additional penalty to its error term. The goal of regularisation is to fit SVM well enough on the training set with some errors, in return for gaining better performance on an unseen test set. This approach avoids *overfitting*, a problem where a classifier seemingly fits excellently on the training set but performs poorly on the test set.

To implement regularisation in SVM, the hyperparameter *C* was introduced. The values for *C* have the following regularisation effects:

- A small C assigns a small penalty for misclassification and *softens* the SVM margin. The result is more misclassifications on the training set. However, these are compensated by better generalisation and performance on an unseen test set.
- A large C assigns a large penalty for misclassification and *hardens* the SVM margin. In other words, it becomes more costly for the classifier to make errors during the training phase. As a result, there are fewer misclassifications on the training set. However, this may lead to the overfitting problem and potentially worse performance on an unseen test set.

Consequently, a practical challenge when training an SVM model is to find the optimal value of C with which the classifier performs reasonably well on both the training and test sets. Let's consider some examples. Continuing the previous example of an outlier near the decision boundary, setting different values for hyperparameter C gives the following effects:

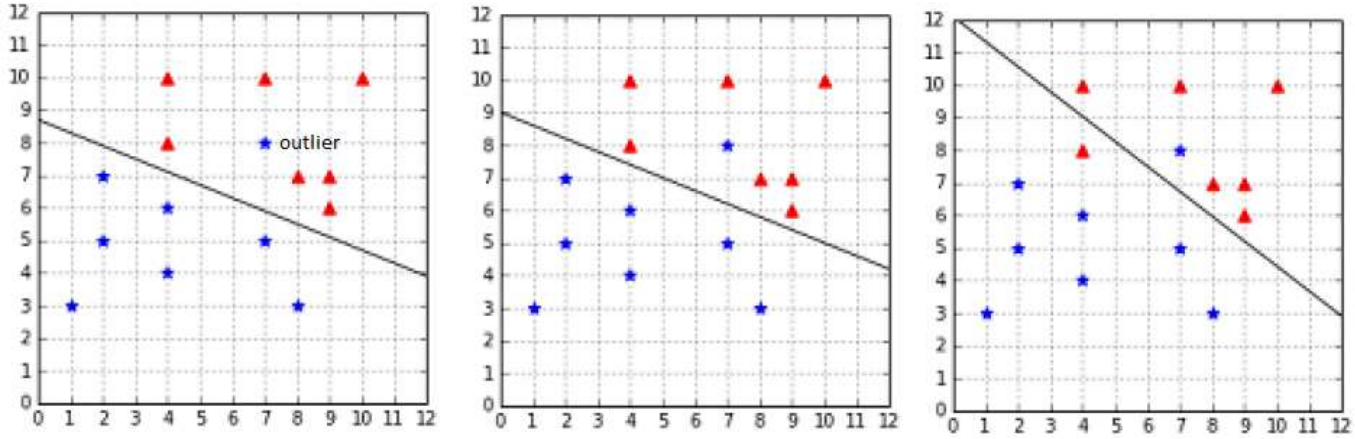


Source: Kowalczyk, A. Support Vector Machines Succinctly.

(from left to right: C=+Infinity, C=1, and C=0.01). In the above, setting C to +Infinity produces a hyperplane identical to the hyperplane produced by a hard margin SVM. Lowering the value to 1 produces a hyperplane with only one misclassification error during the training phase (i.e. the blue star above the hyperplane), in return for a wider margin hyperplane. Further reducing the value of C

introduces a different classification error (the red triangle below the hyperplane). A pragmatic approach would prefer SVM with C=1 *over* C=0.01 because it is more acceptable to misclassify an outlier (the blue star) than to misclassify a non-outlier data point (the red triangle).

For the next problem where an outlier renders the training data to be non-linearly separable, tuning the hyperparameter C has the following effects:



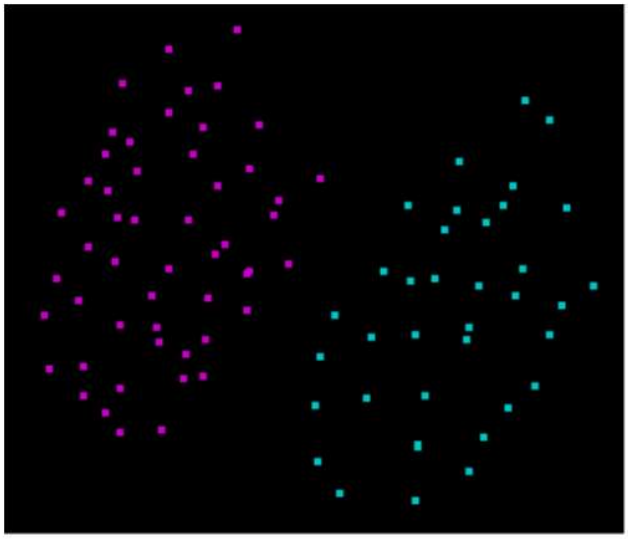
Source: Kowalczyk, A. Support Vector Machines Succinctly.

(from left to right: C=3, C=1, and C=0.01). Here, setting C to either 3 or 1 produces similar hyperplanes because no matter how hard we penalise the errors, the outlying blue star data point always ends up as a misclassification error. We also observe from the rightmost plot that it is not beneficial to further soften the SVM margin by reducing C to 0.01. It only adds additional misclassification errors (the red triangle below the hyperplane) without eliminating the outlier problem. For this situation, sticking to C=3 or C=1 is likely the most optimal solution.

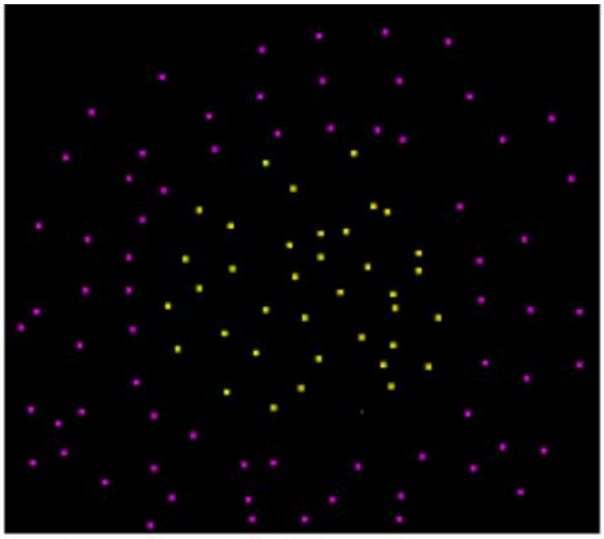
Classifying non-linearly separable data with the kernel trick

As described previously, not all data are linearly separable. This section further deals with this problem.

In the example below, linearly separable data is illustrated on the left. In contrast, non-linearly separable data is shown on the right, for which no straight line exists that cleanly separates the two classes.



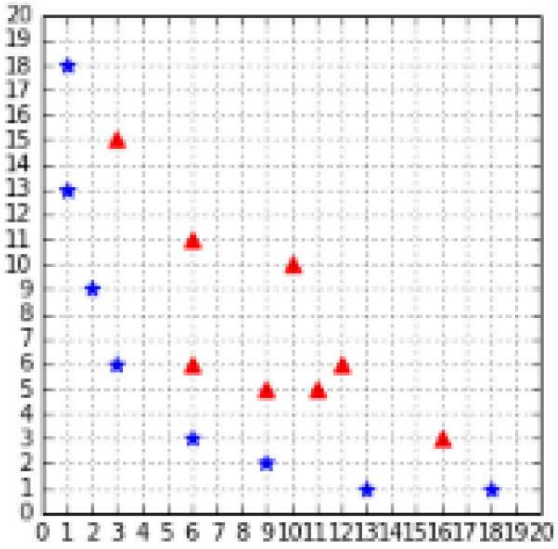
Linearly separable data

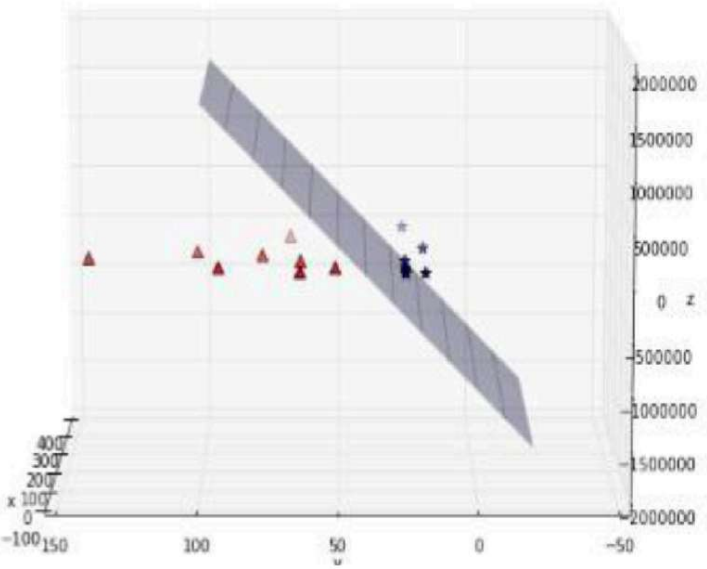


Non linearly separable data

Source: svm-tutorial.com (https://www.svm-tutorial.com/2014/11/svm-understanding-math-part-1/)

The previous soft margin trick is incapable of dealing with non-linearly separable data. Another trick that helps SVM overcome this problem is to transform such data (by applying a specific transformation formula) to be linearly separable in a higher-dimensional space. For example, the dataset below becomes linearly separable by a hyperplane in a 3D space after being transformed from a 2D space:





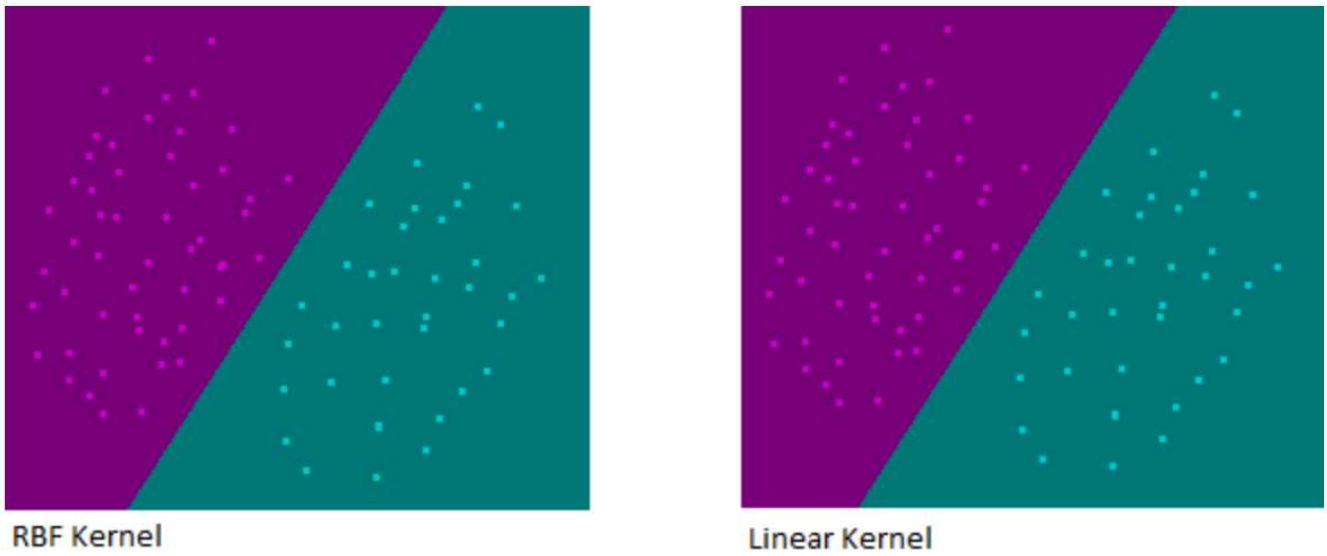
Source: Kowalczyk, A. Support Vector Machines Succinctly.

Unfortunately, such a transformation technique can be computationally expensive. To address this issue, the **kernel trick** was invented for SVM to bypass the transformation process but still achieve the same separation outcome. A kernel is just some mathematical function. Using the right kernel trick depends on choosing the right kernel to use for SVM, explained here.

Linear kernel

The linear kernel is used when data is linearly separable. It is the simplest and the fastest kernel in SVM and does not require a lot of hyperparameters to be adjusted to achieve good performance. It is suitable when there are a lot of features in the dataset (such as in text classification tasks) because (i) texts may be linearly separable, and (ii) applying further transformations on data with that many features rarely yields additional performance.

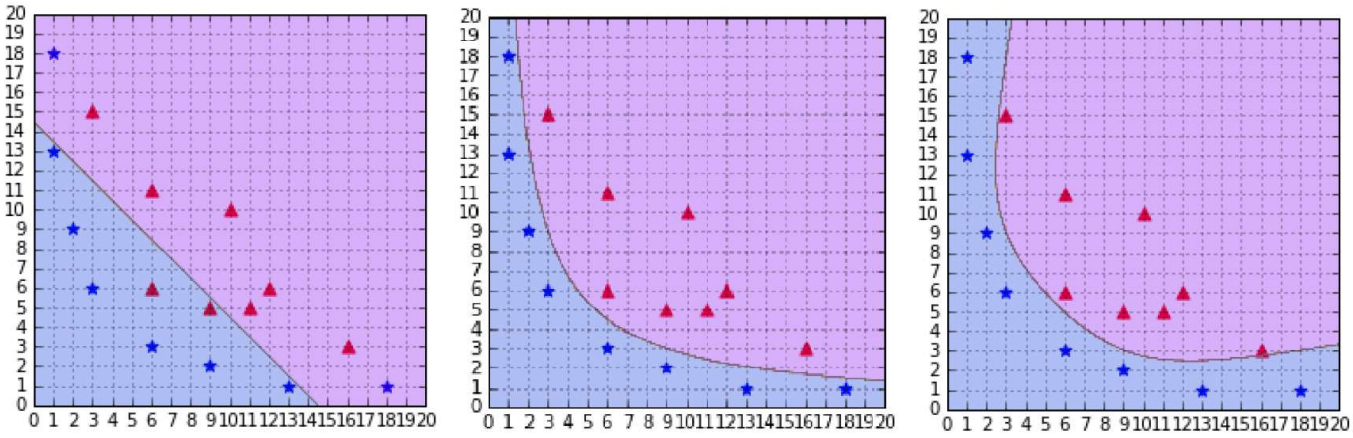
There is little incentive to use other more complicated kernels when the data is linearly separable. In the example below, using the RBF kernel (a sophisticated but slower non-linear kernel) produces the same hyperplane as the linear kernel:



Source: svm-tutorial.com (https://www.svm-tutorial.com/2014/11/svm-understanding-math-part-1/)

Polynomial kernel

The polynomial kernel is suitable for non-linearly separable data. The decision boundary it produces is controlled by a hyperparameter called the *degree* of the kernel. Setting different degrees changes the decision boundaries of SVM. This diagram shows, from left to right, various decision boundaries by the polynomial kernel with degree=1 (identical to linear kernel), degree=2, and degree=6.



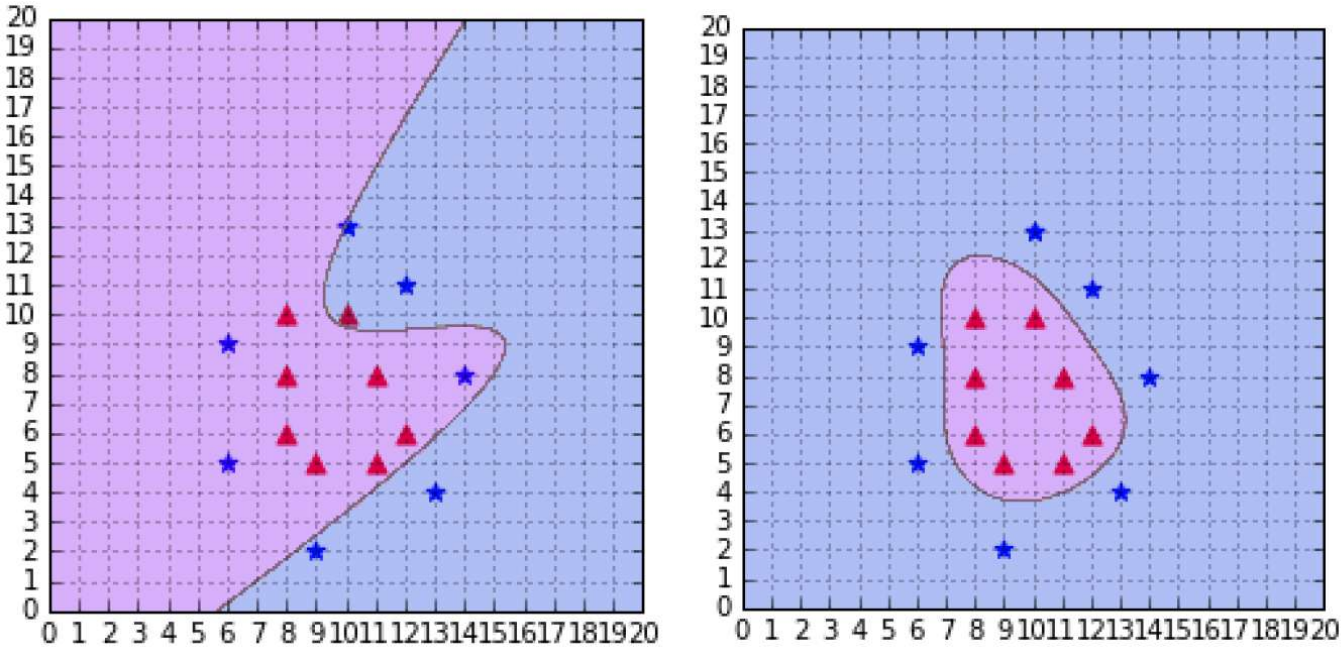
Source: Kowalczyk, A. Support Vector Machines Succinctly.

(from left to right: degree=1, degree=2, degree=6). For the left plot, setting degree=1 is not optimal because the dataset is non-linearly separable and the polynomial kernel produces a similar decision boundary as that of a linear kernel. For the right plot, setting the degree too high (e.g. 6) causes the

decision boundary to overfit the training data so that it may not generalise well to the unseen test set. Finally, in the centre plot, we see that setting degree=2 seems to be an optimal choice as it cleanly separates the two classes without much overfitting.

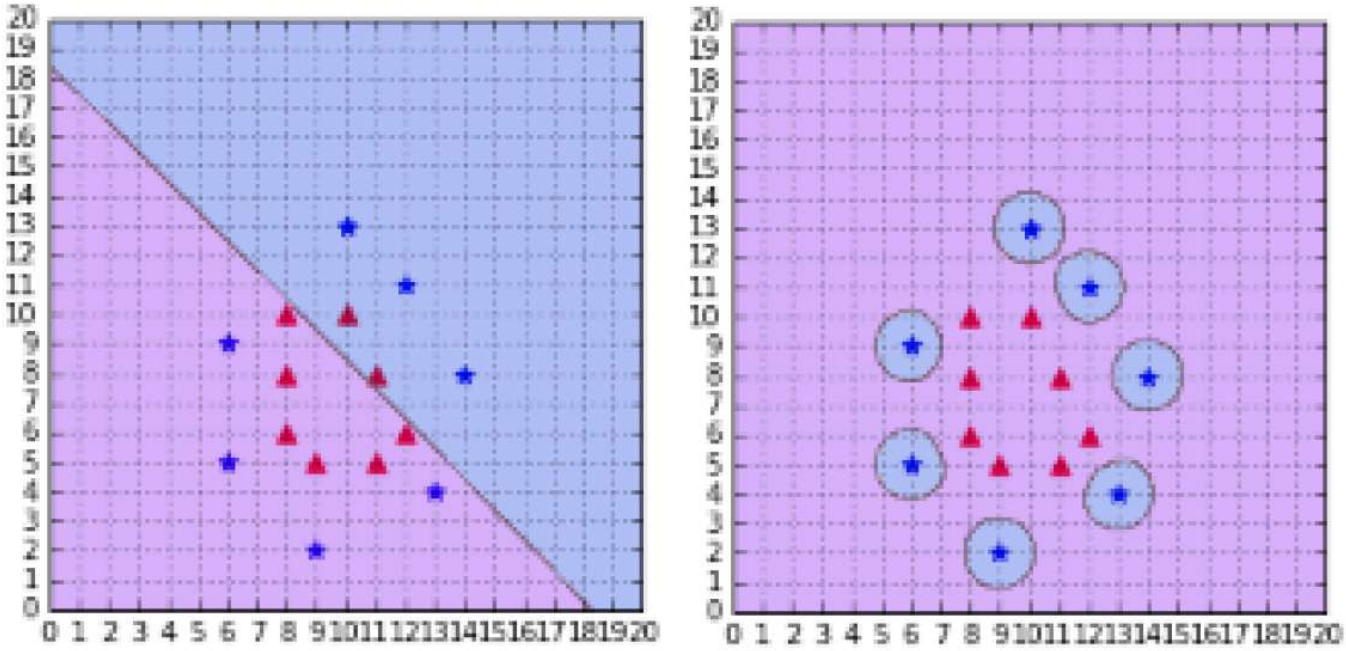
RBF (Radial Basis Function) kernel

There are cases where the polynomial kernel is not good enough, as shown in the left plot below. The RBF kernel can be used to construct a more fitting non-linear decision boundary for SVM (see the plot on the right).



Source: Kowalczyk, A. Support Vector Machines Succinctly.

The behaviour of the RBF kernel, also called the Gaussian kernel, is dictated by a specific hyperparameter called *gamma*. Akin to the polynomial kernel, finding the right gamma value is a matter of trial and error. Choosing a gamma too small produces linear decision boundaries (left plot below). If the gamma is too large, the kernel will overfit the training set (right plot).



Source: Kowalczyk, A. Support Vector Machines Succinctly.

The RBF kernel is the most powerful of all standard SVM kernels. It is a usual practice to use the RBF kernel *first*. This is also why the SVM implementation by the `scikit-learn` has the RBF kernel as the default kernel choice. There are cases where other kernels (linear, polynomial, or others) may perform better than RBF, depending on the nature of the target dataset. A systematic way of searching the right kernel and its appropriate hyperparameter settings (e.g. the grid search) is highly recommended for most modelling scenarios that involve SVM.

Applications

SVM gained its initial popularity in the early 2000s for a handwriting recognition task, where it performed comparably well against the more established neural network algorithm. Subsequently, it became a popular algorithm of choice not just for any general classification but also for document and text classification and image recognition tasks. It is one of the most well-used off-the-shelf supervised machine learning algorithms, often yielding good classification results without complicated tweaks.

SVM has a well-proven mathematical foundation and has been empirically shown to be a reliable classifier. It is also well-known as an effective binary classifier (i.e. it works well for datasets with two classes). It is simple to train and relatively faster than more complex algorithms such as neural networks.

A variant of SVM, Support Vector Regression, is invented to solve regression tasks (i.e. predicting a continuous output variable).

On the other hand, SVM's performance may be limited by the problem of choosing the right kernel function and, depending on the specific SVM libraries you use, may not scale well against extensive, high-dimensional data. It may also not work well when the number of variables (columns) in the dataset

exceeds the number of records (rows).

Finally, an SVM model that relies on default hyperparameter settings does not necessarily give the most optimal performance. Consequently, hyperparameter optimisation is usually needed when using SVM in practice (unlike Naive Bayes which requires little to no hyperparameter optimisation).

9. Overview and concepts

Previous Next

Cross-validation

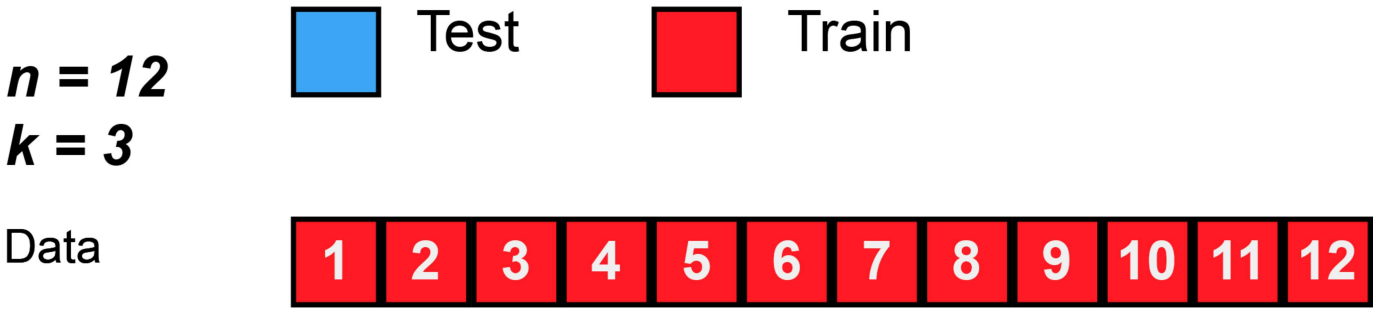
Cross-validation is a resampling procedure for gaining the average estimate of a model's performance. Its objective is to test a model's ability to make predictions on a portion of data that is different from the dataset it was initially trained on.

Cross-validation applies to both regression and classification tasks. The simplest form of cross-validation is the % split between training and test sets (i.e. *one-round* cross-validation). Although we are familiar with this simple cross-validation procedure, there are reasons why it may be insufficient to evaluate a model's performance reliably. The simple % split is known to be prone to the *variability problem*, where a model's performance on the test set is determined more by how the data is split than by the inherent quality of the model. This is evident from the changes we usually observe in a model's performance scores responding to changes in the % or the random seed value.

k-fold cross-validation

To overcome the limitations of the one-round cross-validation, it is recommended to use `k`-fold cross-validation. This technique randomly splits a dataset into `k` equally-sized segments, of which `k`–`1` segments are used for training and the remaining segment is used for testing.

The following animation illustrates k-fold cross-validation for k=3.



Source: wikipedia.com (https://en.wikipedia.org/wiki/Cross-validation_(statistics)#k-fold_cross-validation)

The stepwise procedure is as follows:

1) Shuffle records in the dataset randomly.

2) Split the dataset into `k` segments.

3) For each segment:

- Take the segment as a test data set
- Take the remaining segments as a training data set
- Fit a model on the training set and evaluate it on the test set
- Retain the evaluation score and discard the model

4) Iterate Step 3 `k` number of times until all segments are used as a test set. Summarize the average performance of the model over the iterations.

The common settings for `k` are `k=5` and `k=10`.

Source: machinelearningmastery.com (https://machinelearningmastery.com/k-fold-cross-validation/)

Confusion Matrix

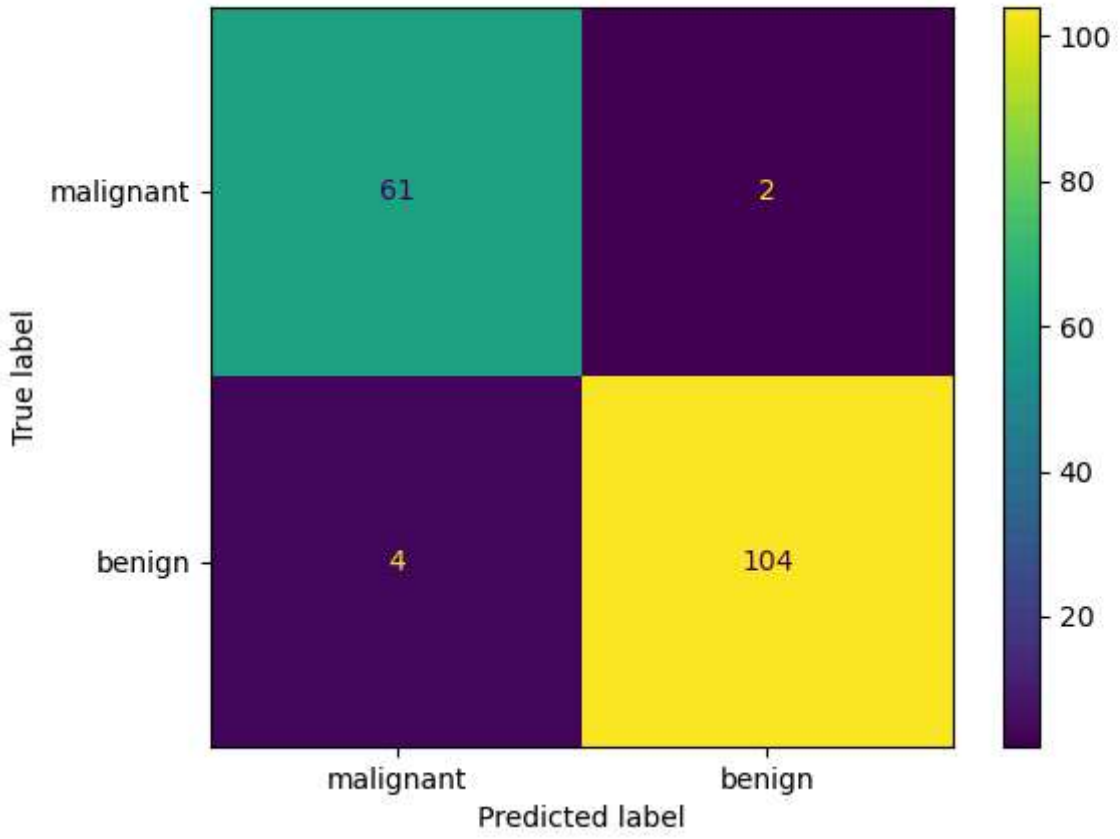
The confusion matrix visually depicts the performance of a ***classifier*** (not for regression) in terms of the number of actual versus predicted class labels. Typically, matrix rows represent the actual class labels, and columns represent the predicted labels. Numbers in the intersections between these rows and columns (i.e. cells) signify the amount of accurate and inaccurate classifications.

Here is the anatomy of the confusion matrix:

		Predicted condition	
	Total population = P + N	Positive (PP)	Negative (PN)
Actual condition	Positive (P)	True positive (TP)	False negative (FN)
	Negative (N)	False positive (FP)	True negative (TN)

Source: wikipedia.com (https://en.wikipedia.org/wiki/Confusion_matrix)

Below is an example confusion matrix for classifying malignant and benign breast cancer tissues.

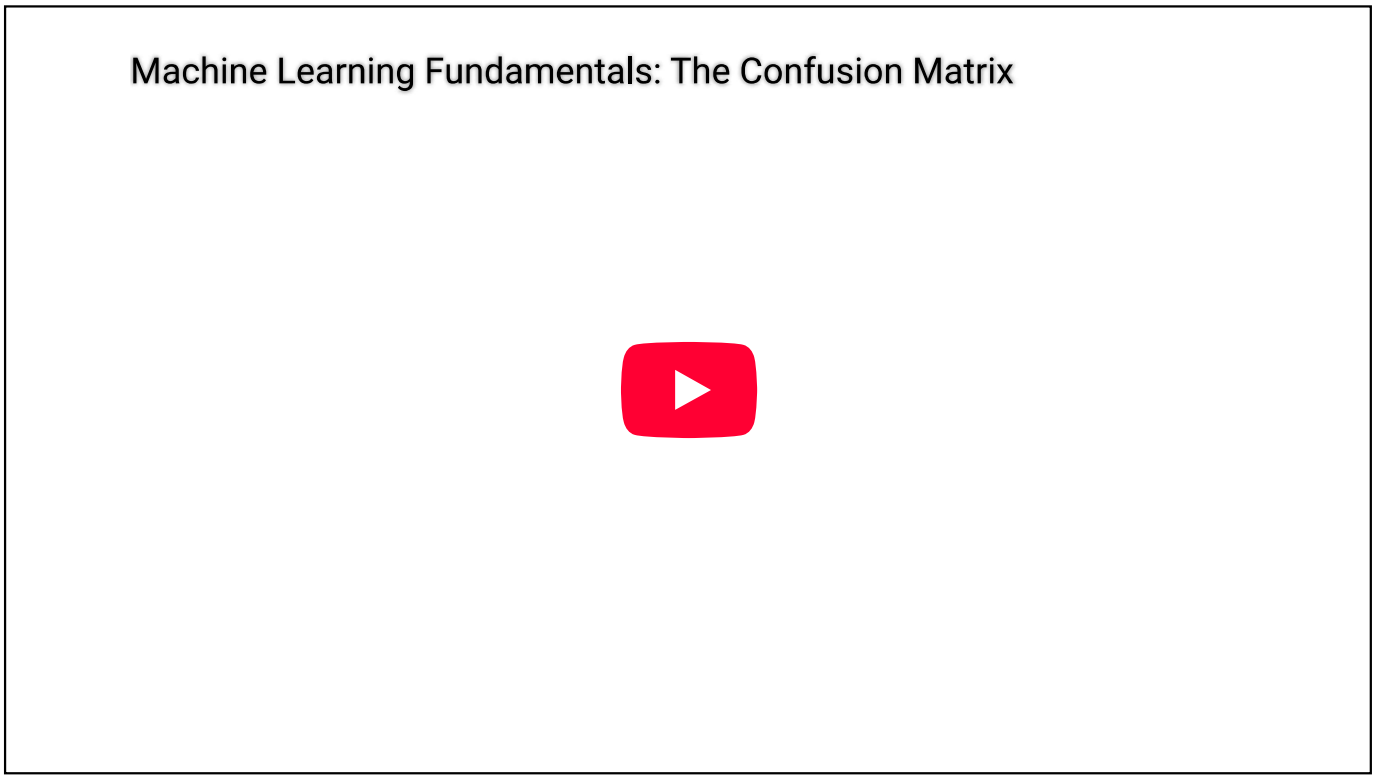


Several important classification metrics can be calculated from a confusion matrix:

- Accuracy
- Recall
- Precision
- F1

Look up the equations of these metrics: https://en.wikipedia.org/wiki/Confusion_matrix (https://en.wikipedia.org/wiki/Confusion_matrix)

The following video further explains the confusion matrix.



Receiver Operating Characteristic (ROC) curve

This video explains the intuition behind the ROC curve.

ROC and AUC, Clearly Explained!

10. Overview and concepts

Previous Next

Ensemble Model: the Random Forests

The term "ensemble" means *a group of things or people acting or being taken together as a whole*[1]. Unlike the Naive Bayes or SVM algorithm which operates with a single predictor or estimator, an ensemble learner combines the predictions of *hundreds to thousands of estimators* built using a pre-specified algorithm. An ensemble model offers better generalisability and/or robustness than a single estimator.

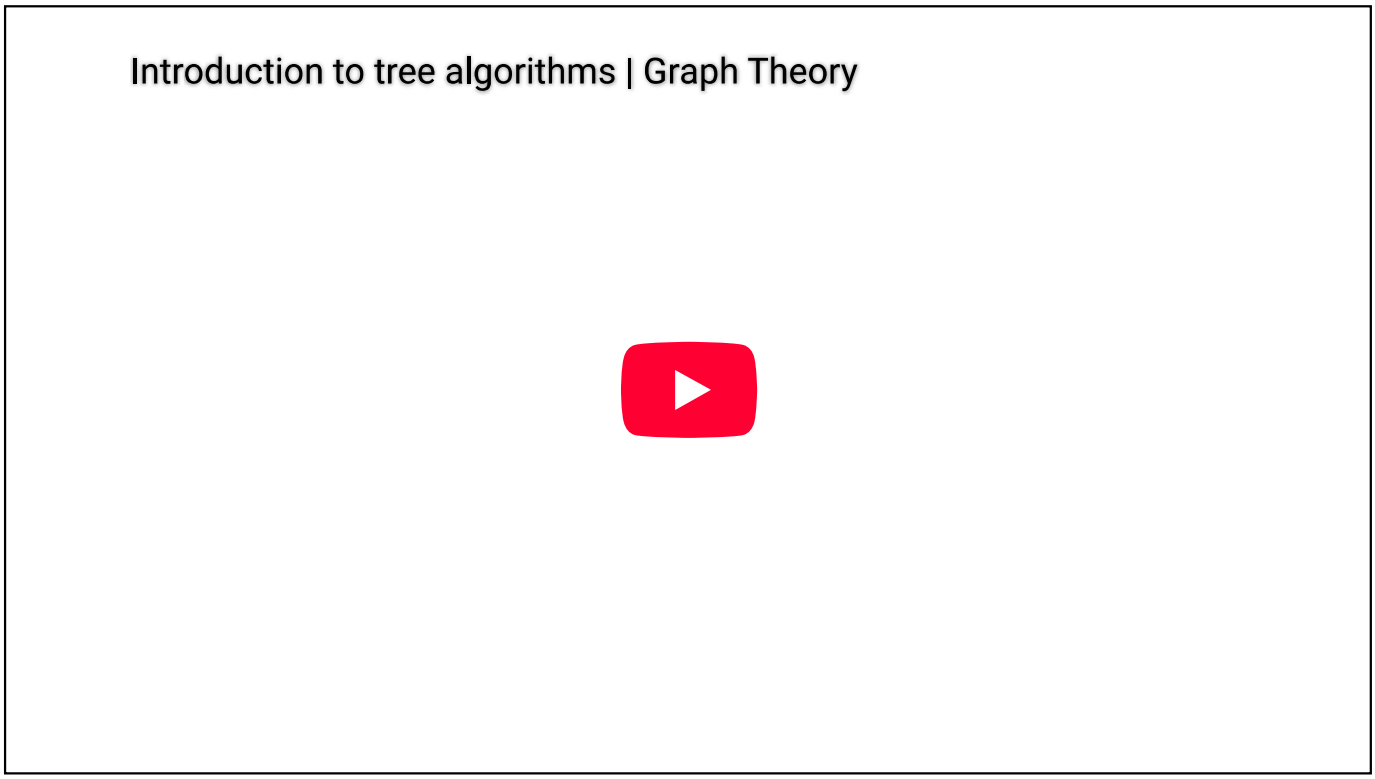
There are various types of ensemble models designed to solve regression and classification problems. One of the most popular ones is the **Random Forests** model. The model works by building multiple estimators based on the *Decision Tree* algorithm. Random Forests can be used to solve regression and classification.

To comprehend the inner workings of the Random Forests algorithm, it is necessary to gain a basic understanding of the Decision Tree algorithm.

[1] *Definition of* ***ensemble*** *from the* ***Cambridge Advanced Learner's Dictionary & Thesaurus*** (https://dictionary.cambridge.org/dictionary/english/) *© Cambridge University Press*

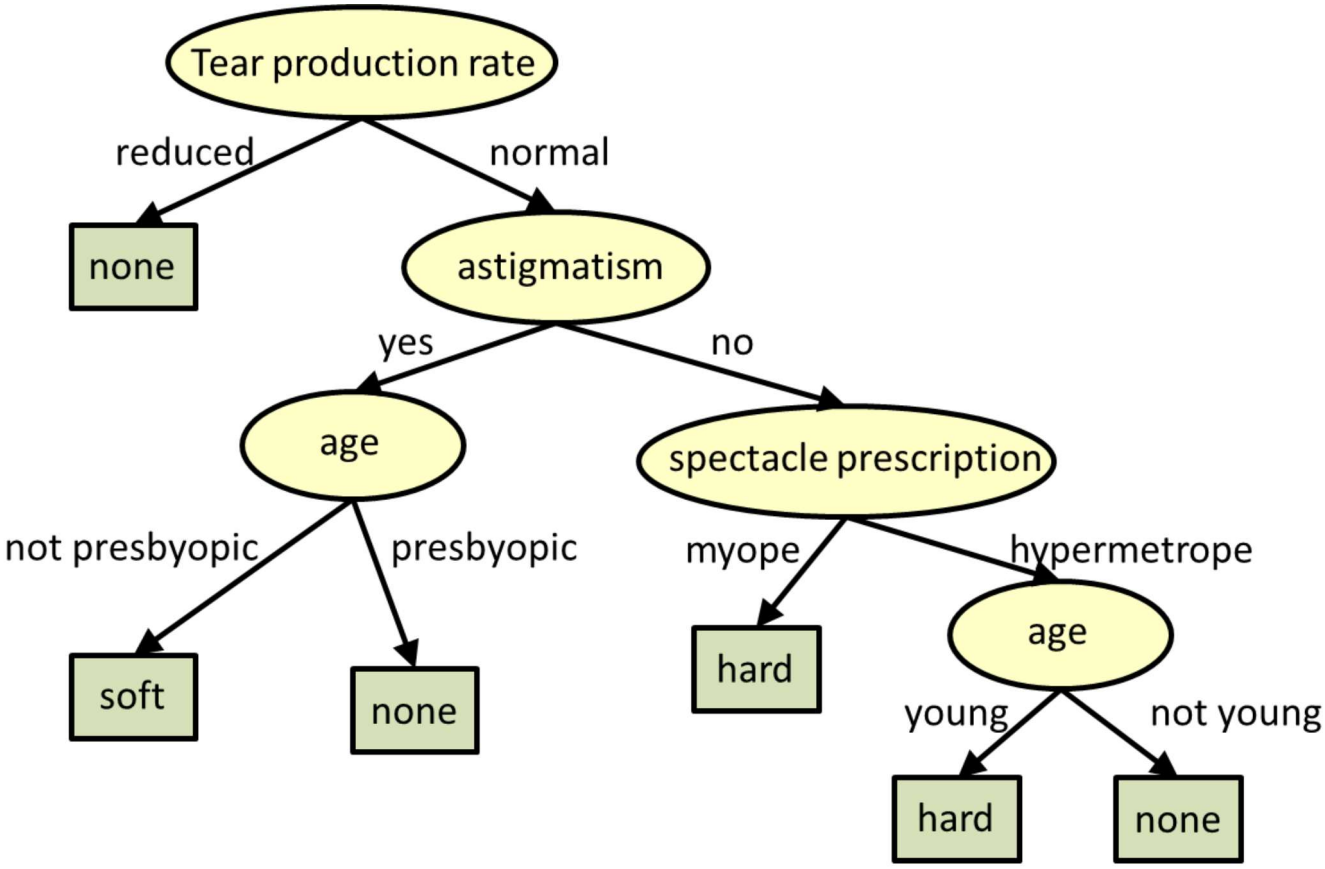
Decision Tree

A decision tree is a tree-like model used for classification and regression. In graph theory, a *tree* is a graph with unique characteristics. In the study of discrete mathematics, data structure, and computer science, the topic of *tree* is a vast subject of its own and will not be discussed in detail here. If you are interested to know more about the tree representation, watch the short video below:



As the name suggests, a decision tree uses a **tree** representation of data to make **decisions**. In data analytics, decisions to be made typically include either (a) assigning a data point to a certain class (*classification*), or (b) assigning a data point with a specific continuous value (*regression*).

Below is an example of a decision tree that determines what kind of contact lens [**soft**, **hard**, **none**] a person should wear. The type of contact lens is the *class* (the response variable). The decision tree then assigns the correct contact lens based on a set of explanatory variables (e.g. tear production rate, spectacle prescription, astigmatism, age) about the person.



Source: Carnegie Mellon University (https://www.cs.cmu.edu/~bhiksha/courses/10-601/decisiontrees/)

A decision tree is trained from a training set, where the learning algorithm recursively learns to build a tree as follows (as described by Raj 2022; Carnegie Mellon University (https://www.cs.cmu.edu/~bhiksha/courses/10-601/decisiontrees/)):

1) Assign all training instances to the root of the tree. Set the current node to the root node.

2) For each attribute:

a. Partition all data instances at the node by the value of the attribute.
b. Compute the information gain ratio from the partitioning.

3) Identify a feature that results in the greatest information gain ratio. Set this feature to be the splitting criterion at the current node. If the best information gain ratio is 0, tag the current node as a leaf and return.

4) Partition all instances according to the attribute value of the best feature.

5) Denote each partition as a child node of the current node.

6) For each child node:

a. If the child node is “pure” (has instances from only one class) tag it as a leaf and return.
b. If not set the child node as the current node and recurse to step 2.

Random Forests: Overview

Relying on a single Decision Tree may not give the most accurate regression or classification results. Random Forests overcomes this limitation by growing *many* decision trees (a forest). To predict the value of a response variable, the algorithm passes the values of explanatory variables down to each tree in the forest. Each tree then gives its predicted value (a 'vote') and Random Forests take the majority vote (alternatively, the mean of predicted continuous values is taken in the case of regression) as the final predicted value.

Each tree is grown as follows (as described by Breiman and Cutler, 2022; University of California, Berkeley (https://www.stat.berkeley.edu/~breiman/RandomForests/cc_home.htm#overview)):

1) If the number of instances in the training set is *N*, sample *N* cases randomly from the original data - but *with replacement*. This sample will be the training set for growing the tree.

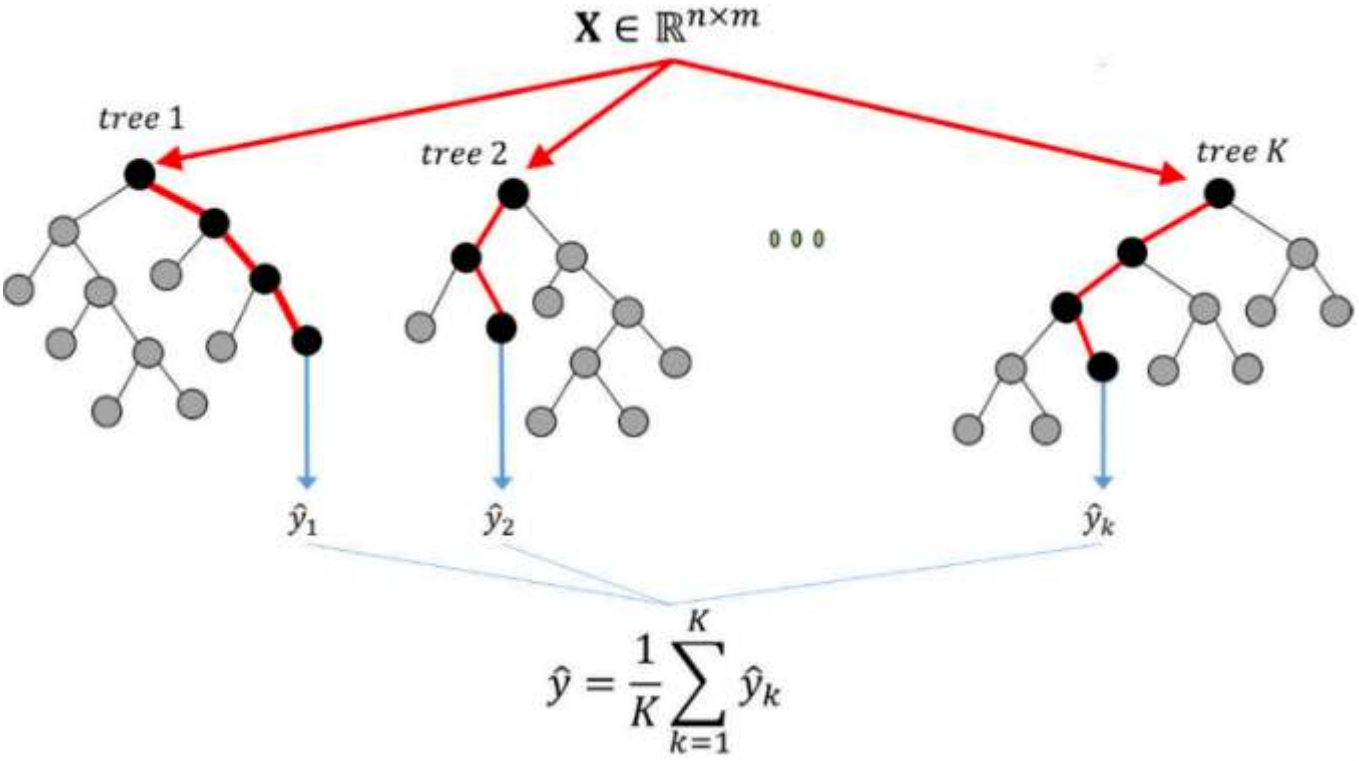
2) If there are *M* explanatory variables, a number $m<M$ is specified such that at each node, *m* variables are selected randomly from the *M* and the best split among these *m* variables is used to split the node. The value of *m* is held constant when growing the forest.

3) Each tree is grown to the largest extent possible. There is no pruning.

This video visually explains the basic workings of the Random Forests model. Can you explain the purpose of bootstrapping, bagging, and feature randomness?

Random Forests for Regression

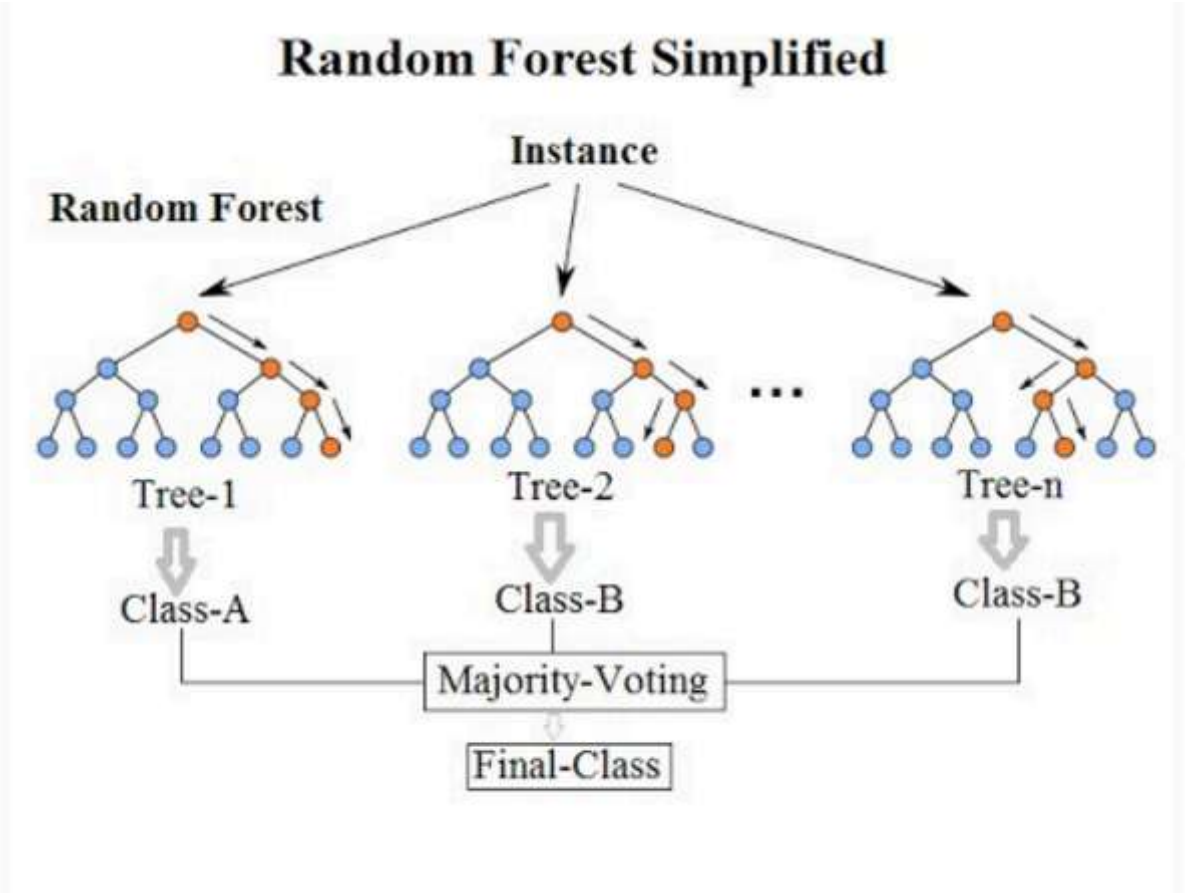
For regression tasks, Random Forests takes the means of all predicted continuous values. The diagram below broadly illustrates the process. In the illustration below, the final prediction $\hat{y}$ is the mean of y values predicted by all trees in the forest.



Source: Aldrich (2020) (https://www.mdpi.com/2075-163X/10/5/420)

Random Forests for Classification

For classification tasks, Random Forests work the same way as regression tasks, except that it picks the class label that receives the most votes by individual trees, as illustrated below.



Source: Wikipedia (https://en.wikipedia.org/wiki/Random_forest)

11. Overview and concepts

Next

Outlier detection

Outliers (or anomalies), by definition, are subsets of data that "do not fit well" with the rest of the data. There are many methods for deciding the manner in which outliers "do not fit well" with the rest of the data. These methods are known as outlier detection algorithms.

There are at least two reasons why you might be interested in detecting outliers in the course of data analysis:

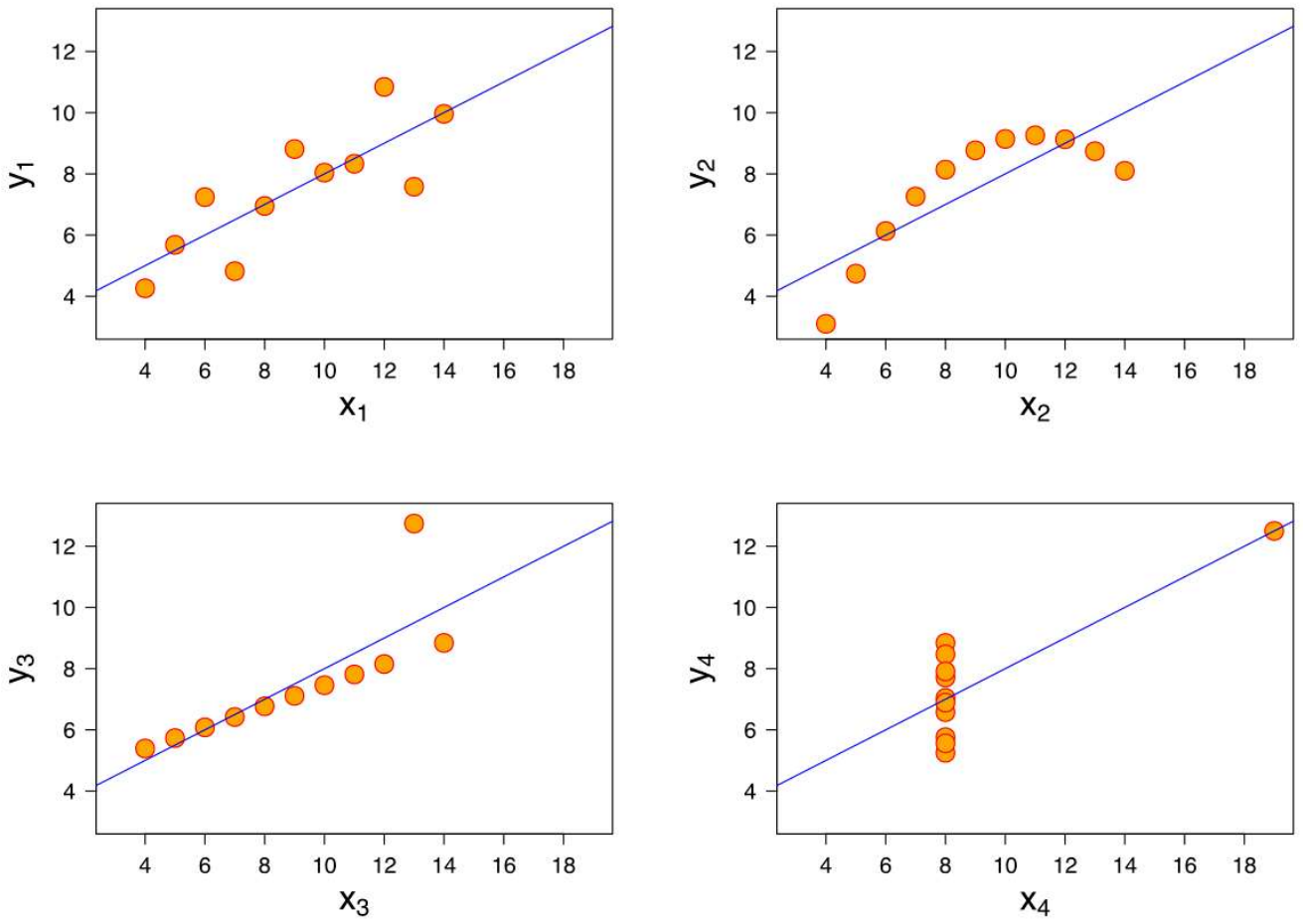
1. Firstly, outliers may represent an extraordinary, unexpected, or unusual phenomenon of certain significance (e.g. credit card fraud, manufacturing defects, etc.).
2. Secondly, outliers in data may adversely affect the quality of a model and therefore must be detected and removed before a model is built. Our lesson in this unit is more concerned with these applications.

Examples from the Anscombe's Quartet

The figure below illustrates Anscombe's Quartet, which is a set of 4 datasets with seemingly identical descriptive statistics:

- Mean of x: 9
- Sample variance of x: 11
- Mean of y: 7.50
- Sample variance of y: 4.125
- Correlation between x and y: 0.816
- Linear regression line: y = 3.00 + 0.500x
- Coefficient of determination of the linear regression: 0.67

When visualised, however, the four datasets possess contrastingly different distributions, as shown in this figure:



The Anscombe's Quartet. Image from Wikipedia (https://en.wikipedia.org/wiki/Anscombe%27s_quartet).

While Anscombe's Quarter is frequently used to illustrate the importance of visualising your data, it also serves as an excellent example of the negative effect that an outlier in data could exert on a model (e.g. linear regression).

The scatterplot at the bottom left of the above figure indicates the presence of an outlying data point. The presence of the outlier is strong enough to skew an otherwise well-fitted linear regression line (the blue line). Consequently, removing this outlier is expected to improve the correlation score between both variables to approximately 1.0 (perfect positive correlation) and correct the linear regression line.

IQR Method

The simplest method for detecting outliers is the Inter-Quartile Range (IQR) method. The IQR method defines an outlier in a variable as:

$$\text{outlier} < Q_1 - 1.5 \times IQR$$

or

$$\text{outlier} > Q_3 + 1.5 \times IQR$$

, where Q_1 and Q_3 are the 25th percentile and the 75th percentile of all values in the variable, and IQR is the difference between Q_3 and Q_1.

Example

The dataset below contains longevity information (typical lifespan) of 40 different mammal species. Is the elephant an outlier according to its longevity?

MammalLongevity.csv

Source: lock5stat.com (https://www.lock5stat.com/datapage2e.html)

To find the answer, perform these calculations:

1. In the longevity variable, we first determine Q_1=8.00 and Q_3=15.25, thus IQR = Q_3 - Q_1=15 - 8 = 7.25.
2. Multiplying the IQR by 1.5 gives 10.875, which we then use to derive the outlier lower limit as 8 - 10.875 = -2.875 years and the outlier upper limit as 15 + 10.875 = 26.125 years.
3. Since the elephant is the only mammal in the dataset with a longevity of 40 years that is greater than the specified outlier upper limit, it is an outlier.

Solution

MammalLongevity.xlsx